ResumeMatch AI — Technical Test Report

Project	ResumeMatch AI (CVisionnaires AI) — CV / job-offer matching platform
Version tested	`main` @ `4c99d1f` (+ test tooling)
Test date	11 October 2026
Environment	Windows 11 · Python 3.14.3 · Django 5.2.18 · DRF 3.18.3 · Node 24.14.1 · React 19 · Vite 8 · SQLite · AI engine in *demo (mock) mode*
Tools	Django test runner + Coverage.py · Vitest · Playwright · Locust · Lighthouse
Design	UML class, use case and activity diagrams (PlantUML) — section 3
Prompts	Development prompts, embedded LLM prompts, performance, adaptation, and model choice — section 4

1. Executive summary

The full application (Django REST API + React SPA) was tested with **five complementary tools**, each covering a different level of the test pyramid: backend unit/integration tests, frontend unit tests, browser end-to-end tests, load testing and a quality audit.

Indicator	Result
Automated functional tests executed	**86** (52 backend + 19 frontend + 15 end-to-end)
Functional tests passed	**86 / 86 (100 %)**
Backend code coverage	**83 %** of statements (2 169 statements, 369 missed)
Frontend unit coverage	**90 %** of `src/lib` business logic — **4.4 %** of the whole `src` tree
API under normal load (20 users)	**975 requests, 0 errors**, median **7 ms**, p95 **23 ms**
API under stress (100 users)	**4 517 requests**, median **10 ms**, p95 **97 ms**; the only errors (3.4 %) are intentional `429` quota responses
Lighthouse (desktop)	Performance **89–98**, Accessibility **100**, Best practices **100**, SEO **82–83**
Lighthouse (mobile landing page)	Performance **65** (LCP 7.1 s)

Verdict: the application is functionally stable — every user journey works end to end, the API is fast and did not produce a single server error under load, and accessibility/best-practice audits are perfect. The

main improvement areas are **mobile loading performance of the landing page**, **SEO metadata**, **unit tests for React components**, and the **cost of login under concurrent load**.

---

2. System under test

```
Browser (React 19 SPA, Vite)  ──/api──►  Django 5 + DRF (JWT auth)  ──►  SQLite
                                                  │
                                                  ├── AI service (Anthropic/OpenAI/Gemini, or dete
                                                  └── Job feeds (Remotive, Arbeitnow, Jobicy, Adzu
```

Module	Backend app	Main frontend pages
Authentication	`accounts`	Landing, Login, Register, Profile
CV management	`cvs`, `pdf_service`, `ats_service`	Analysis wizard (upload step)
Analysis engine	`analyses`, `ai_service`, `scoring`	Wizard, Results, History, Compare, Dashboard
Job search	`job_search_service`, `job_import_service`	Job search, Application board
In-app interview	`analyses` (`ApplicantInterview`, `Interview`)	Interviews
Application email	`outreach`	Email appliance, Results
Administration	`adminpanel`	Admin console

Test data: the database was seeded with the built-in `seed_demo` command (1 admin, 3 candidates — Sarah, Karim, Léa — with CVs, analyses and cover letters). The developer database was backed up before testing and restored afterwards.

AI mode: no LLM API key is configured, so the AI engine ran in its deterministic *demo* mode. Response times therefore measure the application itself, not a third-party AI provider.

---

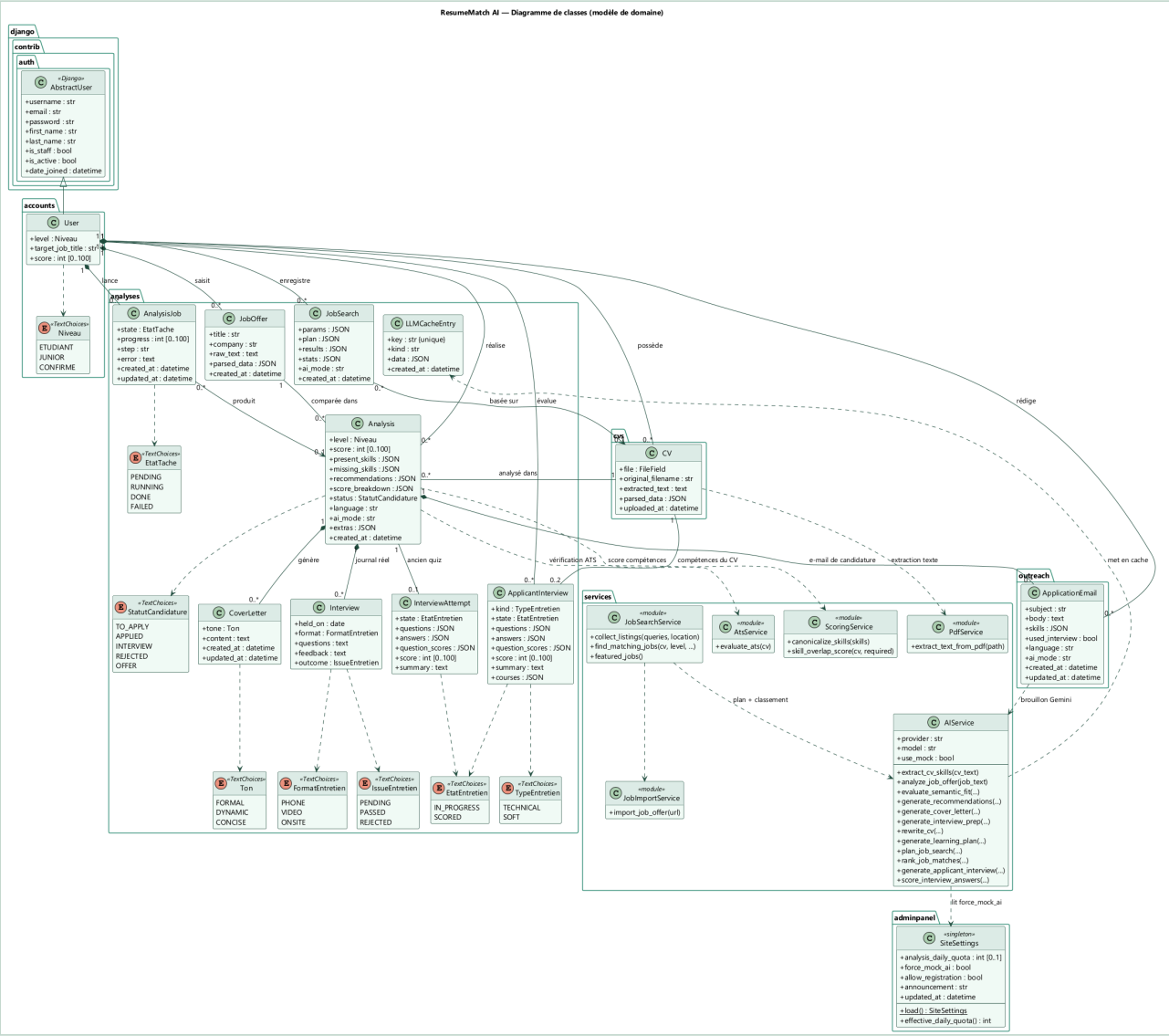
3. Software design — UML diagrams

The application was modelled with **PlantUML** from its actual code (Django models, API views, services and React pages). Diagram labels are in French, the project's working language. Sources are in `docs/conception/diagrammes/`; PNG and SVG renderings are in `docs/conception/images/`.

Diagram	UML type	What it shows
Domain model	Class	Entities, attributes, enumerations, multiplicities and service dependencies

Diagram	UML type	What it shows
Visitor and Candidate	Use case	26 use cases with include / extend relations and two external systems
Administration	Use case	Admin console features, all requiring an `is_staff` account
Run a CV / job analysis	Activity	Upload, quota check, asynchronous job and progress polling
AI job search	Activity	Search plan, 5 job sources in parallel, filtering and AI ranking
Registration, login and JWT session	Activity	Sign-up, login, automatic token refresh and logout
In-app interview and application email	Activity	CV-skill questions, errors and practice points, then the six-part email

3.1 Class diagram

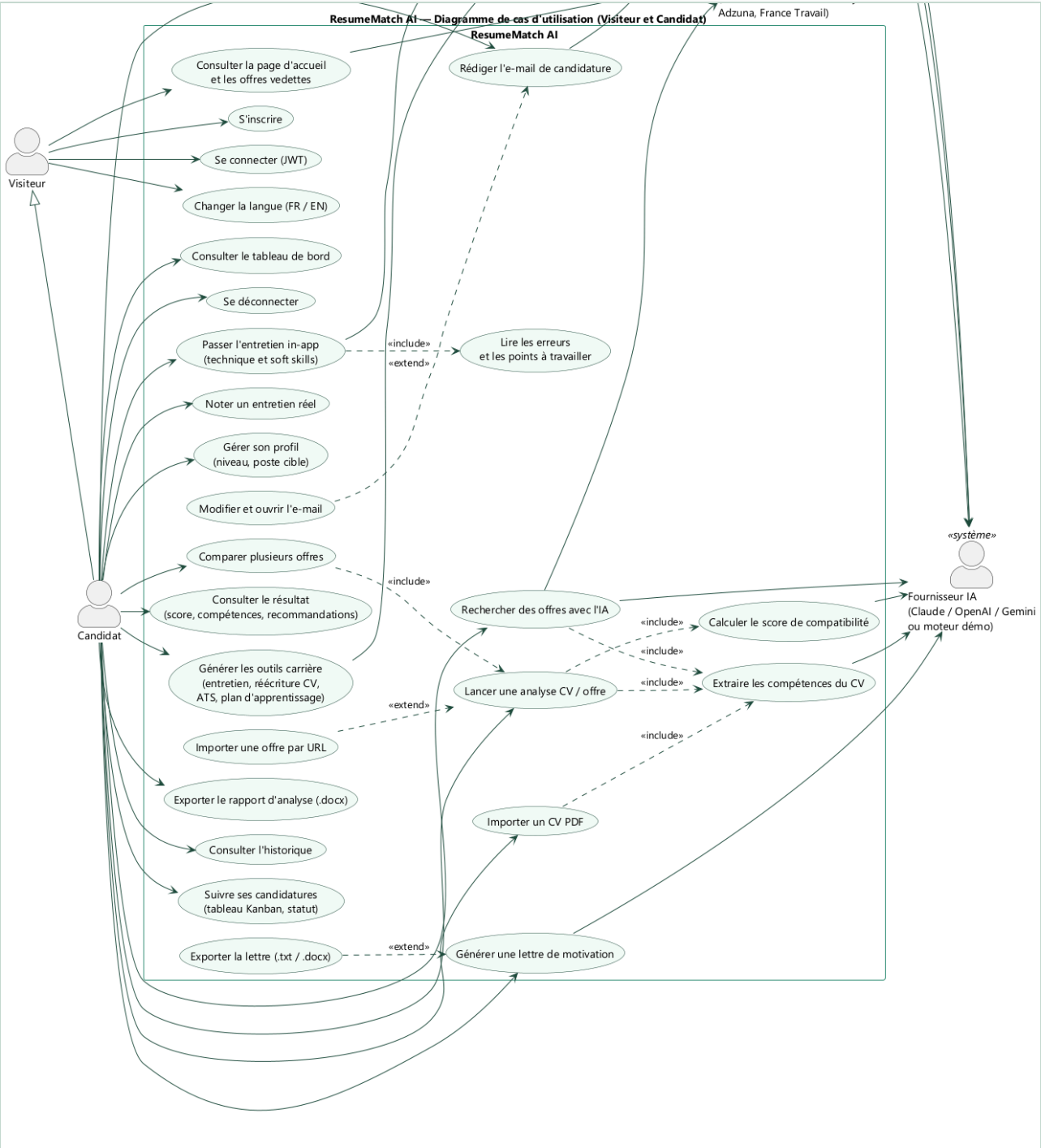


Class	Role
`User`	Extends Django's `AbstractUser` with a level (student / junior / senior), a target job title and a main score filled from the in-app interviews

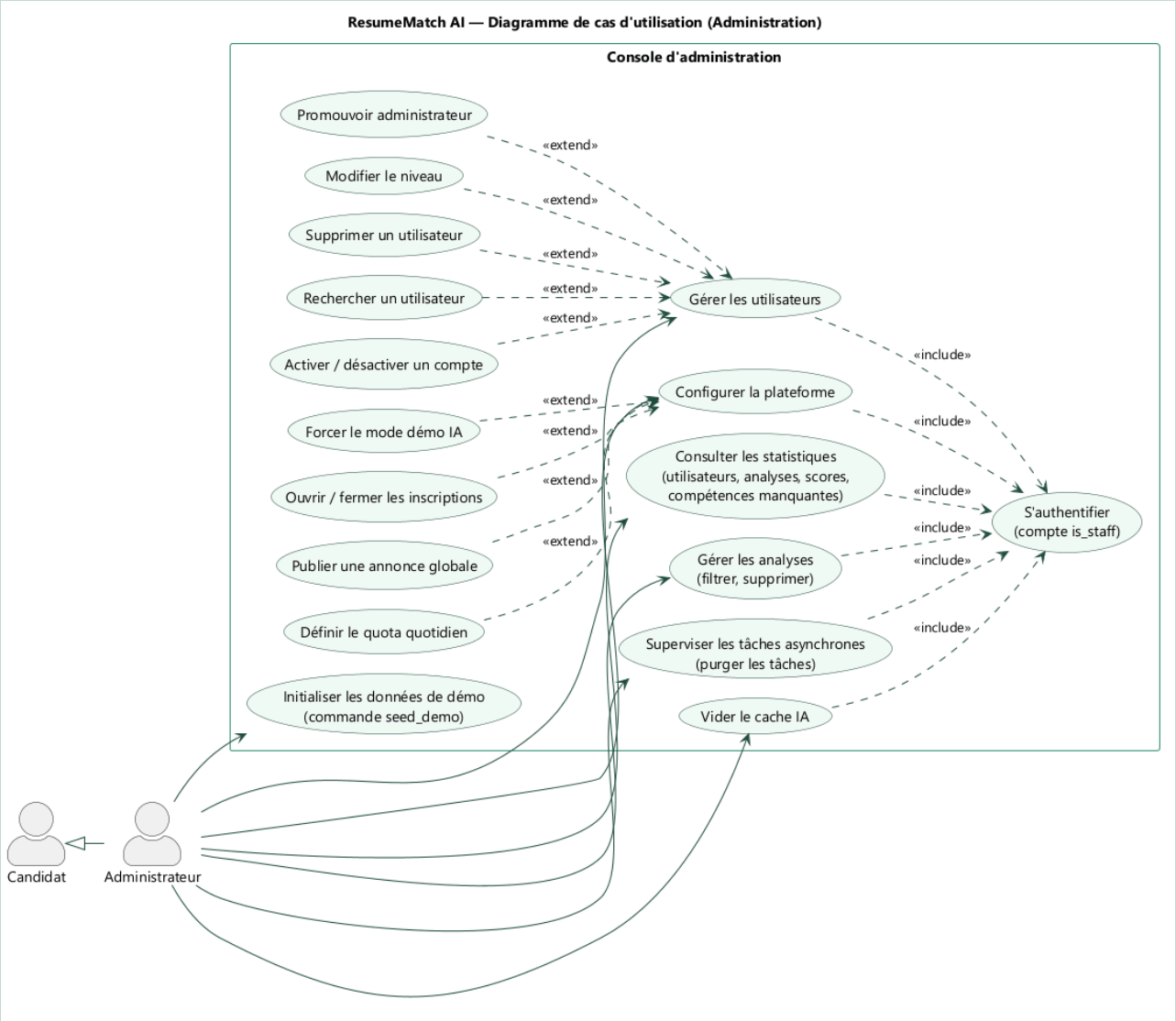
Class	Role
`CV`	Uploaded PDF, extracted text and structured skills (`parsed_data`)
`JobOffer`	Job offer pasted or imported from a URL, with its structured analysis
`Analysis`	Matches one CV with one offer: score 0–100, present / missing skills, recommendations, application status (Kanban) and career-toolkit results (`extras`)
`CoverLetter`	Cover letter generated for an analysis in a given tone
`ApplicantInterview`	CV-only interview, technical or soft skills: one question per skill retrieved from the CV, written answers, a 0–100 score, the errors in each answer and what to practise next. Independent of any job application. The mean of the scored interviews is the user's main score
`Interview`	Manual log of a real interview (date, phone / video / on site, notes, outcome). It does not change the application status or any score
`InterviewAttempt`	Older quiz tied to one analysis. Submitting it can still blend 20 % of its score into that job-match score. The Interviews page does not use it
`ApplicationEmail`	Editable application email for one analysis: subject, body, CV skills, quoted interview answers, language, and whether Gemini or the local draft wrote it
`AnalysisJob`	Asynchronous analysis task (pending → running → done / failed) with progress
`JobSearch`	AI job search: parameters, plan and ranked results (last 10 kept per user)
`LLMCacheEntry`	Cache of LLM answers, keyed by a hash of the prompt
`SiteSettings`	Configuration singleton: daily quota, forced demo mode, registrations, announcement
`AIService`	Provider-agnostic AI service with Pydantic validation and mock fallback

A user **owns** (composition, `on_delete=CASCADE`) their CVs, offers, analyses, jobs, searches and application emails. An analysis links exactly one CV and one offer, while a CV or an offer can be reused in many analyses. An `AnalysisJob` produces at most one analysis, and a `JobSearch` is based on at most one CV (both `SET_NULL`). A CV owns at most two `ApplicantInterview` rows, one technical and one soft. An analysis owns many real `Interview` notes and at most one `InterviewAttempt` .

3.2 Use case diagrams



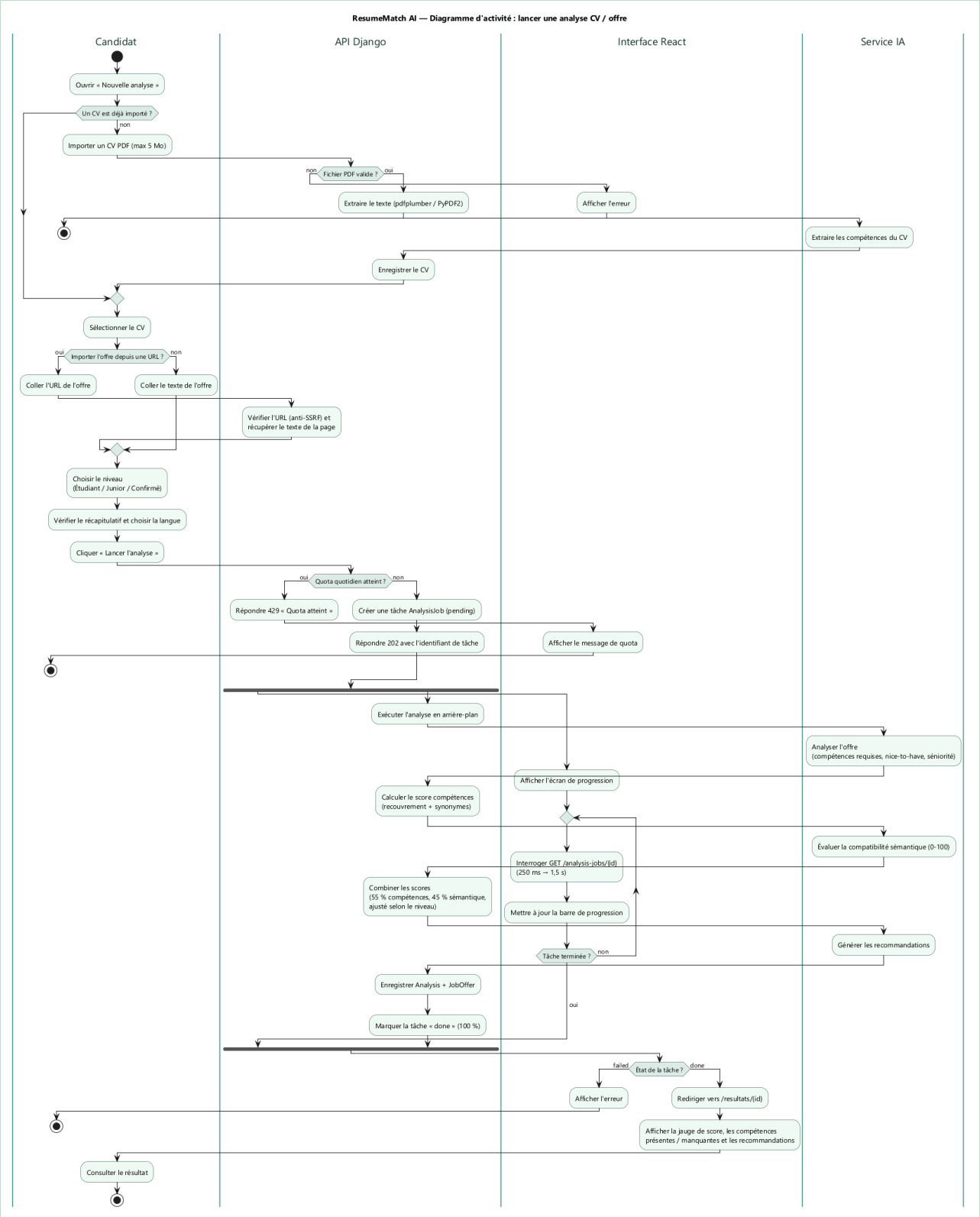
The **Visitor** can browse the landing page, register, log in and switch language. The **Candidate** inherits these and uses every analysis feature. *Run an analysis* includes skill extraction and score calculation; *import an offer from a URL* extends it; *export the cover letter* extends cover-letter generation. *Take the in-app interview* is separate from any application: the candidate answers one technical interview and one soft-skills interview built from the skills on the uploaded CV; submitting updates only that interview score and the user's main score. *Log a real interview* records date, format and outcome without changing the application. *Write an application email* drafts a six-part message from the CV, the job and any scored interview, then lets the candidate edit, save and open it in the mail client. The AI provider (Claude, OpenAI, Gemini or the demo engine) and the job sites (Remotive, Arbeitnow, Jobicy, Adzuna, France Travail) are external system actors. Gemini writes the application email when a key is set.



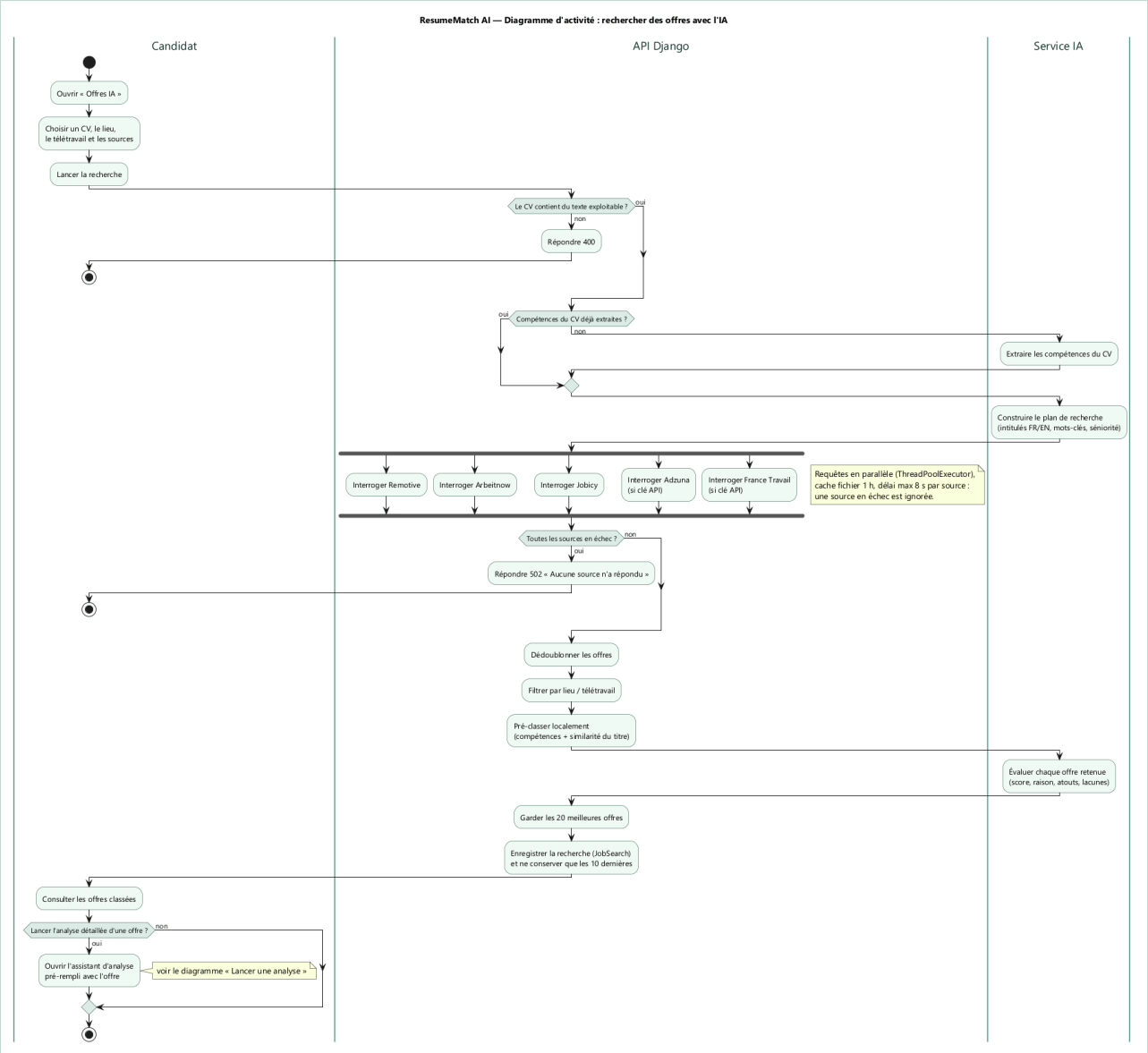
The **Administrator** inherits from the Candidate. Every administration use case includes authentication with an `is_staff` account. User management (search, activate, promote, change level, delete) and platform configuration (quota, demo mode, registrations, announcement) are modelled as extensions.

3.3 Activity diagrams

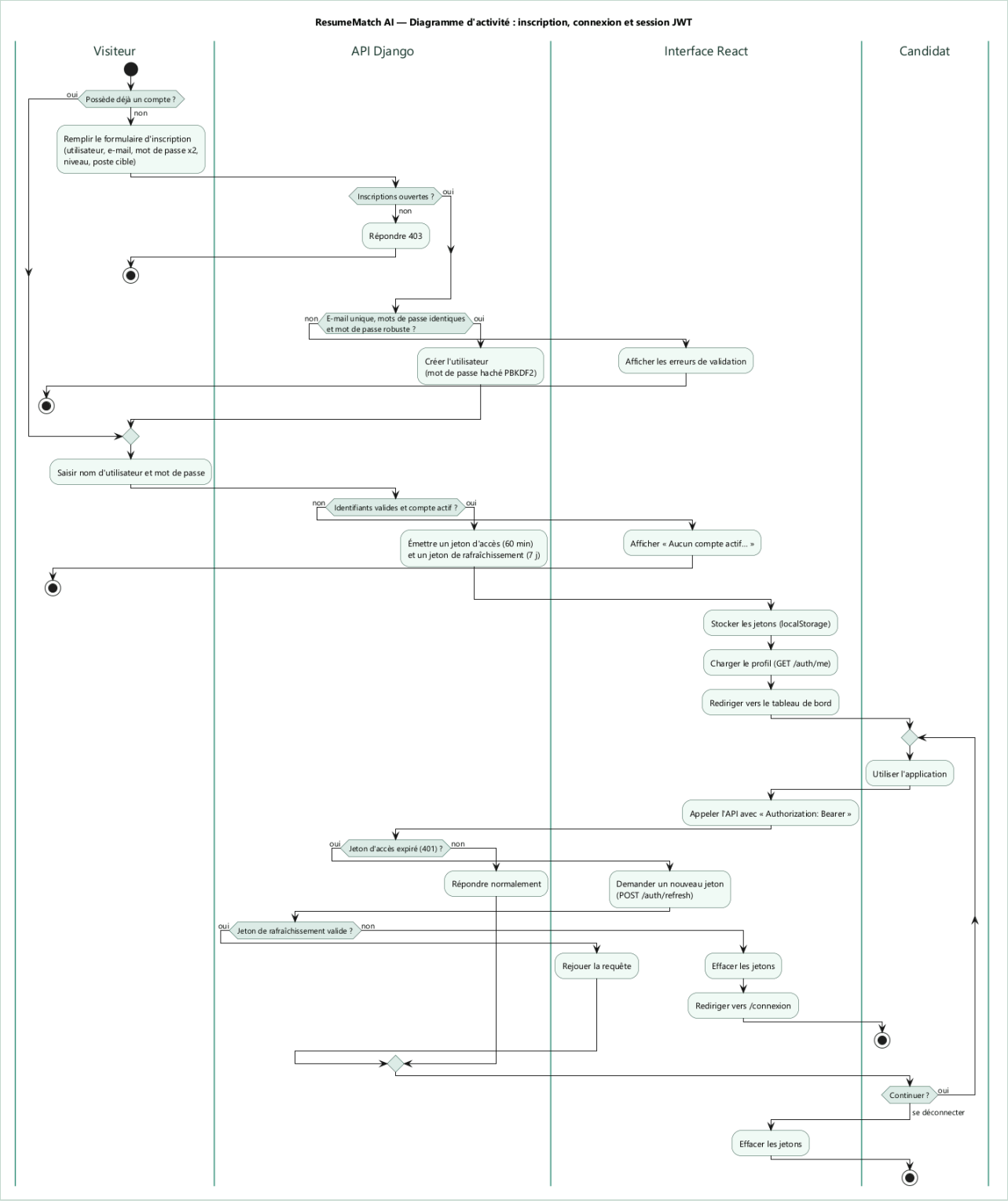
Swimlanes show who performs each action: the candidate, the React interface, the Django API or the AI service.



The CV is validated (PDF, 5 MB maximum) and its skills extracted; the offer is pasted or imported from a URL (SSRF-protected). If the daily quota is reached the API answers **429**; otherwise it creates an `AnalysisJob` and answers **202** immediately. The analysis then runs in the background (offer analysis, skill score, semantic score, 55 % / 45 % weighting adjusted to the level, recommendations) **in parallel** with the interface polling the job (250 ms growing to 1.5 s), before redirecting to the results.



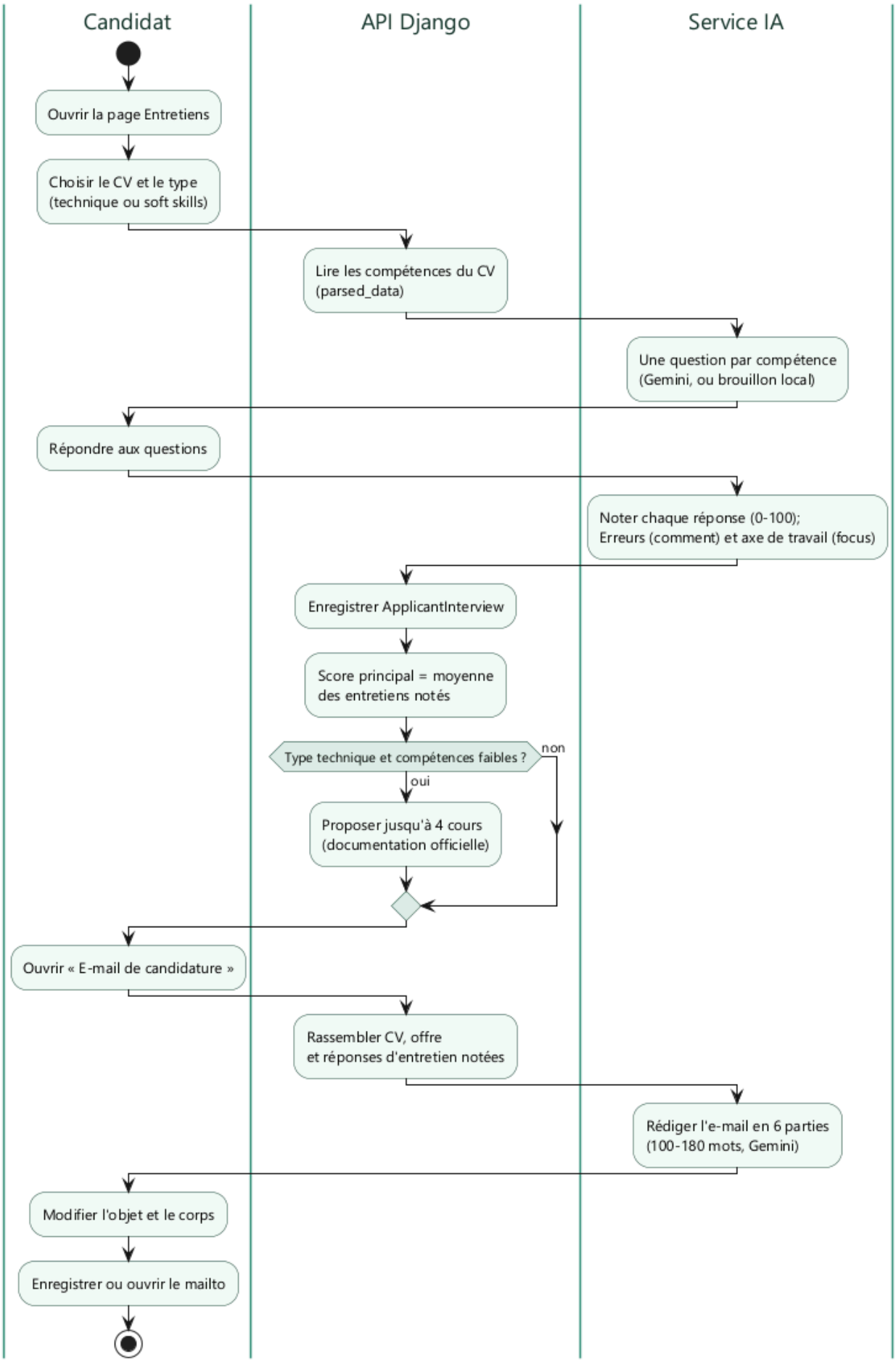
The API rejects a CV without usable text (**400**), reuses already-extracted skills, asks the AI for a search plan (French and English titles, keywords, seniority), then queries the **5 sources in parallel** (1-hour cache, 8 s maximum per source, a failing source is skipped; **502** if all fail). Offers are deduplicated, filtered by location / remote, pre-ranked locally and ranked by the AI; the top 20 are kept and the search is saved.



Registration checks that sign-ups are open (**403** otherwise) and validates the form (unique e-mail, matching and strong passwords; PBKDF2 hashing). Login issues an access token (60 min) and a refresh token (7 days). When an API call returns **401**, the interface requests a new token and replays the request, or clears the session and redirects to the login page if the refresh token is invalid.

3.4 In-app interview and application email

ResumeMatch AI — Diagramme d'activité : entretien in-app et e-mail de candidature



These two flows are on the class diagram, the candidate use-case diagram and the activity diagram above. Swimlanes are the candidate, the Django API and the AI service.

In-app interview. From the Interviews page the candidate opens a CV. The API builds two interviews, technical and soft skills, with one question per skill found in `parsed_data` (up to 20). Answering and submitting scores each answer from 0 to 100. Gemini returns the mistakes in that answer (`comment`) and what to practise next (`focus`). The page lists the weakest points first. The result is stored on `ApplicantInterview` and does not change any job-match score. A scored technical interview also lists up to four official-documentation courses for the weakest skills. The user's main score is the rounded mean of the scored interviews. A separate list at the bottom of the page keeps notes from real interviews (`Interview`); those notes do not change a score.

Application email. From Email appliance, or from the results of one analysis, the candidate generates a draft. The prompt requires one purpose, 100–180 words, and this order: subject, greeting, reason for writing, context from real CV skills and interview answers, a request to discuss, thanks, then the signature. The candidate can edit the subject and body, save them, and open a `mailto:` link. With a Gemini key the draft is marked `ai_mode=gemini`; with no key the local six-part draft is marked `mock`. The email quotes scored `ApplicantInterview` answers when they exist, otherwise the older `InterviewAttempt`.

---

4. Prompts used to build the application

ResumeMatch AI was built with the **Cursor AI agent**. Two kinds of prompts are documented here: the instructions given to the agent during development, and the prompts the application itself sends to the LLM. The full French version, with every prompt text, is in `docs/conception/PROMPTS.md`.

4.1 Project timeline

Date	Commit	Author	Step
05/10/2026	`47f5c5f`	azouzi marwen	Repository initialised
05/10/2026	`58adf16`	Cursor Agent	Full MVP: Django API + React interface
05/10/2026	`34bf48f`	Cursor Agent	History search filter fix
05/10/2026	`1ad595b`	Cursor Agent	Scoring unit tests (synonyms, overlap)
05/10/2026	`fbc695e`, `d6d6a07`	Cursor Agent	Forest / teal colour palette and favicon
05/10/2026	`c10ac4a`	Cursor Agent	`start-local.sh` one-command launcher
05/10/2026	`13a2769`	Mahdibouazizi + Cursor	AI job search, admin console, demo data, career toolkit, async analyses, FR / EN
11/10/2026	`4c99d1f`	brouri12 + Cursor	Interface responsiveness optimisation
11/10/2026	—	Cursor Agent	5 module branches, test campaign, this report, UML design

4.2 Creation phase — 5 October 2026 (reconstructed prompts)

> *The conversations of this phase were held with a Cursor agent on another account (commits signed Cursor Agent, co-author `marwenazouzi44`), so their original text is **not available locally**. The prompts below are **reconstructed** from the exact content of each commit; they describe faithfully what was requested and delivered, but are not word-for-word.*

#	Commit	Reconstructed prompt (summary)
C1	`58adf16`	Build the full-stack « ResumeMatch AI » app (Django 5 + DRF + JWT, React + Vite + Tailwind): JWT accounts with level, PDF CV upload and text extraction, LLM skill extraction, job-offer analysis, hybrid 0–100 score (55 % skills / 45 % semantic, adjusted by level), recommendations, cover letters in 3 tones with export, history; provider-agnostic AI with an offline mock when no API key is set
C2	`34bf48f`	Fix the history page search filter
C3	`1ad595b`	Add unit tests for scoring: skill synonyms and overlap score
C4	`fbc695e`, `d6d6a07`	Replace the cream / terracotta palette with forest / teal tones and align the favicon
C5	`c10ac4a`	Add `start-local.sh`: venv, migrations, Django on :8765 and Vite on :43124 in one command
C6	`13a2769`	Add AI job search over real job feeds, live offers on the landing page, admin console, `seed_demo`, interview prep, CV rewrite, ATS check, learning plan, .docx report, async analyses, URL import, multi-offer comparison, Kanban board, score timeline, FR / EN interface, LLM cache, throttling and quotas

4.3 Evolution and quality phase — 11 October 2026 (original prompts)

Exact prompts typed in Cursor, in order (automatic environment notifications excluded):

#	Original prompt (verbatim)	Result
P1	`https://github.com/brouri12/resumematch-ai pull the project and run it in ide cursor`	Project cloned and running locally
P2	`i want you to split this progect to 5 modules and the you push each module in that repo git in braches that has the name of each module ok`	Branches `auth`, `cv-management`, `analysis`, `job-search`, `admin-panel`
P3	`now i want you to solve the prolems in the app because it take alot of time to respense for any action`	Slowness traced to page animations (API was fast); navigation ≈ 0.35 s instead of over 1 s — commit `4c99d1f`
P4	`YES`	Fix pushed to `main` and the 5 module branches
P5	`I WANT YOU TO use 5 tools of test to test the full app and the collect the resultes and make a comparation table and dont forget to take screenshots to add it tho the technical raport that you gonna make it ok`	Test campaign and this report (sections 5 to 13)

#	Original prompt (verbatim)	Result
P6	`AJOUTER LES PROMPT utilisuer durant touts la creation de app et creerer les diagramme de classe et usecase et activiter`	Sections 3 and 4 of this report, `docs/conception/`
P7	`ajouter le travail demander au rapport technique`	Design and prompts integrated into this report

4.4 Prompts embedded in the application

The application sends its own prompts to the LLM (`backend/services/ai_service.py`). Each prompt demands an answer **« UNIQUEMENT en JSON strict »**, includes the expected JSON schema (generated from a Pydantic model), and is validated with up to **3 attempts** before falling back to the mock engine. Three reusable fragments personalise them:

Fragment	Values	Example of injected text
Level	`etudiant`, `junior`, `confirme`	« Le candidat est JUNIOR. Mets en avant les réalisations concrètes, outils maîtrisés et le travail en équipe. »
Tone	`formal`, `dynamic`, `concise`	« Ton concis, direct, sans fioritures (max 250 mots). »
Language	`fr`, `en`	« Write all textual content in English. »

Method	Feature	Prompt instruction (beginning)
`extract_cv_skills`	CV parsing	« Extrais du CV les informations suivantes en JSON strict. »
`analyze_job_offer`	Offer analysis	« Analyse cette offre d'emploi et extrais les compétences et infos en JSON strict. »
`evaluate_semantic_fit`	Semantic score	« Évalue la compatibilité sémantique entre le CV et l'offre (0-100). »
`generate_recommendations`	Recommendations	« Propose des recommandations d'amélioration du CV pour cette offre. »
`generate_cover_letter`	Cover letter	« Rédige une lettre de motivation personnalisée. »
`generate_interview_prep`	Interview prep	« Prépare le candidat à l'entretien pour cette offre. Génère 8 à 10 questions probables… »
`rewrite_cv`	CV rewrite	« Sélectionne 4 à 8 phrases/puces du CV qui gagneraient à être réécrites… sans inventer de faits »
`generate_learning_plan`	Learning plan	« Construis un plan d'apprentissage pour combler ces compétences manquantes… URLs réelles uniquement »
`plan_job_search`	Job search plan	« Tu prépares une recherche d'offres d'emploi pour ce candidat. Déduis du CV : target_titles… »
`rank_job_matches`	Offer ranking	« Évalue l'adéquation entre le CV et CHAQUE offre… Sois exigeant : une offre hors métier du candidat doit être sous 30. »

Method	Feature	Prompt instruction (beginning)
`generate_applicant_interview`	In-app interview questions	« Prépare un entretien qui évalue la personne. Ce n'est pas une candidature à un emploi. » One question per CV skill, technical or soft
`score_interview_answers`	Interview feedback	« Note chaque réponse d'entretien de 0 à 100. » `comment` states the errors; `focus` states what to work on
Email draft in `outreach/services.py`	Application email	« Rédige un e-mail de candidature professionnel, un seul objectif : postuler. » Six parts, 100–180 words, no invented facts. Sent to Gemini

Prompt-engineering practices applied: structured output with schema + Pydantic validation; tolerant JSON extraction and retries; anti-hallucination constraints (« texte EXACT du CV », « sans inventer de faits », « URLs réelles uniquement »); cost control through text truncation (4 000–12 000 characters), the `LLMCacheEntry` cache, quotas and throttling.

4.5 Performance improvements

The October campaign showed that the API itself was already fast (median 7–10 ms). The delay a user felt came from the interface. Commit `4c99d1f` shortened page animations so navigation fell from over 1 s to about **0.35 s**. The same design choices keep later features (interviews, emails) from blocking the browser.

Technique	Where	Effect
Analysis returns **202** and runs in the background	`AnalysisJob`	The wizard does not wait for the full AI call; the page polls every 250 ms, backing off to 1.5 s
Job-feed cache, 1 hour, 8 s cap per source	`job_search_service`	A failed source is skipped; public feeds are not called on every search
LLM answer cache	`LLMCacheEntry`, hash of provider + model + prompt	A repeated prompt is not sent again
Truncation of CV and offer text	4 000–12 000 characters	Each model call stays bounded
Quotas	20 analyses / 24 h, AI routes 30/hour	Load tests received **429**, not server errors
Access token 60 min, refresh 7 days	JWT	Users rarely repeat the slow login (0.8 s alone, 1.8 s at 100 users)

Interview questions are built from the skill list already stored on the CV, so opening the Interviews page does not re-parse the PDF. Email generation is one JSON call, then the draft is edited locally.

4.6 Adaptation techniques

The same engine changes its wording and its weights from the candidate's profile, the language of the screen, and the kind of task.

Signal	Adaptation
Level `etudiant` / `junior` / `confirme`	A level paragraph is injected into the prompt. The semantic part of the match score is multiplied by **0.7 / 1.0 / 1.2**, then the 55 % / 45 % weights are normalised again
Language `fr` / `en`	Prompt fragment plus the React catalogues. Emails, interview questions and feedback follow the language of the page
Cover-letter tone	`formal`, `dynamic` or `concise`
Interview kind	`technical` asks one question per technical skill on the CV; `soft` asks one question per soft skill. No company name and no job title
Known recipient	The email greeting uses the name on the offer when it exists, otherwise « Dear Hiring Manager, » or « Madame, Monsieur, »
Scored interview	The email may quote those answers. If none exist, it falls back to the older per-job `InterviewAttempt`
No key, invalid JSON, or admin « force demo »	The deterministic mock returns a valid object. Tests run with an empty key so they never call a provider
Interview vs job match	Submitting the in-app interview does **not** change `Analysis.score`. It updates `User.score`, the rounded mean of the scored technical and soft interviews

4.7 Choice of AI models

The two new modules use the same configured model as the rest of the engine. Interview questions and interview grading share one model, and the application email uses that same model. The user's main score is the average of the technical and soft interviews, so the two interviews are not graded by different models. Only the prompt changes.

Module	Call	What Gemini returns
In-app interview	`generate_applicant_interview`	One question per skill on the CV. Technical skills stay in the technical interview. Soft skills stay in the soft-skills interview. No company and no job title
Interview feedback	`score_interview_answers`	A score from 0 to 100, `comment` (the errors in that answer) and `focus` (what to practise). The page lists the weakest points first
Application email	Draft in `outreach/services.py`	Subject and body, 100–180 words, six parts, facts taken only from the CV, the offer and the scored interview. The candidate can still edit the text. `ai_mode` is `gemini` when this call succeeds, `mock` when it does not

Model	Role
Gemini (`gemini-flash-lite-latest`)	Running model for analysis, both interviews and the application email. Free tier, OpenAI-compatible endpoint, live JSON call returned **HTTP 200**
Claude Sonnet	Default name in `.env.example`. Strong at JSON and written answers. Not the running model: after a small trial credit the API is paid
Deterministic mock	Used when `LLM_API_KEY` is empty, when the admin forces demo mode, or after three failed JSON attempts. Sections 6–9 measured this engine, not a provider

Gemini is called through `https://generativelanguage.googleapis.com/v1beta/openai` with `response_format: json_object`. Temperature for those calls is **0.2**.

4.8 API keys and model settings

Keys live in the project `.env` file, which is git-ignored. The report does not copy secret values. An empty main key selects the mock engine for every module, including interviews and emails.

Variable	Meaning
`LLM_PROVIDER`	`gemini` for the running app. Also accepted: `anthropic`, `openai`, `compatible`
`LLM_API_KEY`	Key for Gemini. Empty = demo mode on interviews, emails and analyses
`LLM_MODEL`	`gemini-flash-lite-latest`
`LLM_BASE_URL`	Empty for Gemini. Set only for another OpenAI-compatible host
`LLM_MAX_RETRIES`	Attempts before the mock fallback (default 3)
`SiteSettings.force_mock_ai`	Admin switch that ignores the Gemini key

---

5. Test strategy

Level	Question answered	Tool
Unit + integration (backend)	Do models, services, API endpoints and permissions behave correctly? How much code is exercised?	**Django test runner + Coverage.py**
Unit (frontend)	Is the client-side business logic (scoring display, i18n, helpers) correct?	**Vitest** (+ V8 coverage)
End-to-end (system)	Can a real user complete every journey in a real browser?	**Playwright**
Performance / load	How does the API behave with many simultaneous users?	**Locust**
Quality audit	Is the UI fast to load, accessible, secure and SEO-friendly?	**Lighthouse**

---

6. Tool 1 — Django test runner + Coverage.py (backend)

Purpose: unit and integration tests of the REST API, business services and permissions, measured with statement coverage.

Command

```
 cd backend
python -m coverage run --source=accounts,cvs,analyses,adminpanel,services `
  --omit="*/migrations/*,*/tests*.py" manage.py test -v 2
python -m coverage report --sort=cover
python -m coverage html -d ../docs/test-report/coverage-backend-html
```

Results

Metric	Value
Tests run	52
Passed / failed / errors	**52 / 0 / 0**
Duration	12.9 s
Statement coverage	**83 %** (1 800 / 2 169)

Least-covered modules	Coverage
`cvs/views.py`	45 %
`services/pdf_service.py`	53 %
`cvs/serializers.py`	59 %
`accounts/serializers.py`	63 %
`analyses/views.py`	65 %

Best-covered: `scoring.py` 95 %, `ats_service.py` 95 %, `analyses/models.py` 94 %, `adminpanel/views.py` 90 %, all serializers of `analyses` and `adminpanel` 100 %.

Covered behaviours include: JWT registration/login, admin-only endpoints (`non_staff_is_forbidden`), daily analysis quota, async analysis jobs, DOCX report export, LLM JSON-retry fallback, ATS layout detection, SSRF protection on URL import (`private_addresses_are_rejected`), job-feed caching and failure isolation.

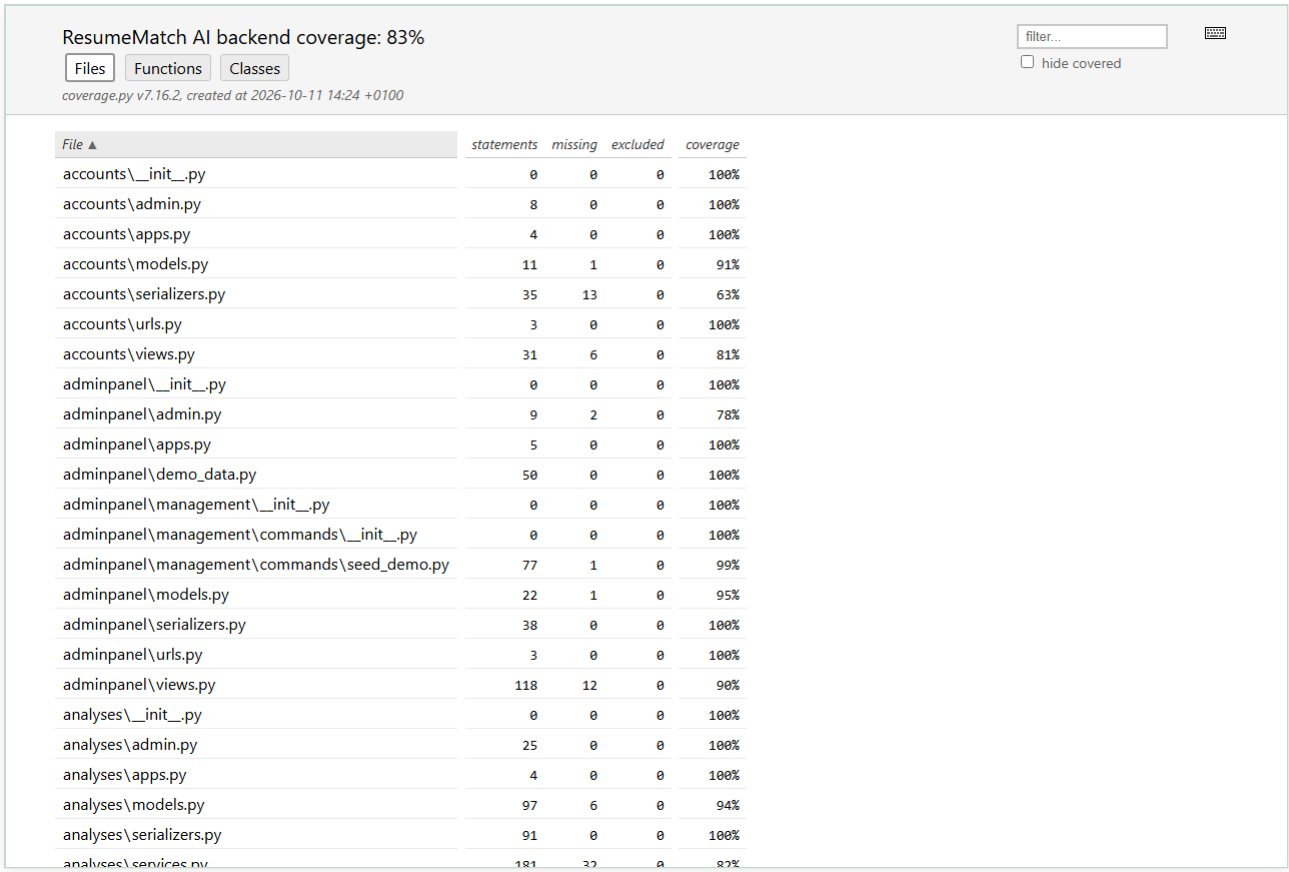
> *The red lines in the console capture are **expected log messages** produced by tests that deliberately simulate failures (an unreachable job source, an invalid LLM response); the tests themselves pass.*

```
PS C:\resumematch-ai> python manage.py test -v 2 (coverage run)

test_clear_cache (adminpanel.tests.AdminPanelTests.test_clear_cache) ... ok
test_delete_user_cascades (adminpanel.tests.AdminPanelTests.test_delete_user_cascades) ... ok
test_force_mock_overrides_api_key (adminpanel.tests.AdminPanelTests.test_force_mock_overrides_api_key) ... ok
test_jobs_list_and_purge (adminpanel.tests.AdminPanelTests.test_jobs_list_and_purge) ... ok
test_me_exposes_is_staff (adminpanel.tests.AdminPanelTests.test_me_exposes_is_staff) ... ok
test_non_staff_is_forbidden (adminpanel.tests.AdminPanelTests.test_non_staff_is_forbidden) ... ok
test_overview_aggregates (adminpanel.tests.AdminPanelTests.test_overview_aggregates) ... ok
test_registration_can_be_closed (adminpanel.tests.AdminPanelTests.test_registration_can_be_closed) ... ok
(adminpanel.tests.AdminPanelTests.test_seed_creates_personas_with_varied_statuses) ... ok
(adminpanel.tests.AdminPanelTests.test_seed_is_idempotent_and_reset_recreates) ... ok
test_user_search_and_toggle_active (adminpanel.tests.AdminPanelTests.test_user_search_and_toggle_active) ... ok
(adminpanel.tests.PdfBuilderTests.test_generated_pdf_is_readable_with_accents_and_pages) ... ok
(analyses.tests.AnalysisApiTests.test_async_job_completes_and_exposes_analysis) ... ok
test_compare_creates_one_job_per_offer (analyses.tests.AnalysisApiTests.test_compare_creates_one_job_per_offer) ... ok
(analyses.tests.AnalysisApiTests.test_create_analysis_records_mock_mode_and_language) ... ok
test_daily_quota_blocks_extra_analyses (analyses.tests.AnalysisApiTests.test_daily_quota_blocks_extra_analyses) ... ok
test_extras_are_generated_then_cached (analyses.tests.AnalysisApiTests.test_extras_are_generated_then_cached) ... ATS
test_import_url_endpoint (analyses.tests.AnalysisApiTests.test_import_url_endpoint) ... ok
(analyses.tests.AnalysisApiTests.test_other_users_cannot_read_jobs_or_extras) ... ok
test_report_export_is_docx (analyses.tests.AnalysisApiTests.test_report_export_is_docx) ... ok
test_system_status_reports_mock_mode (analyses.tests.AnalysisApiTests.test_system_status_reports_mock_mode) ... ok
test_unknown_extra_kind_is_404 (analyses.tests.AnalysisApiTests.test_unknown_extra_kind_is_404) ... ok
test_generation_is_not_cached (services.tests_features.AIServiceLiveTests.test_generation_is_not_cached) ... ok
(services.tests_features.AIServiceLiveTests.test_invalid_json_retries_then_falls_back_to_mock) ... LLM JSON validation
(services.tests_features.AIServiceLiveTests.test_job_parsing_is_cached_after_first_live_call) ... ok
test_language_is_passed_to_prompt (services.tests_features.AIServiceLiveTests.test_language_is_passed_to_prompt) ... ok
test_clean_cv_scores_high (services.tests_features.ATSTests.test_clean_cv_scores_high) ... ok
test_column_detection (services.tests_features.ATSTests.test_column_detection) ... ok
(services.tests_features.ATSTests.test_keyword_match_requires_word_boundaries) ... ok
test_problematic_layout_is_penalized (services.tests_features.ATSTests.test_problematic_layout_is_penalized) ... ok
(services.tests_features.JobImportTests.test_html_fallback_skips_navigation_and_scripts) ... ok
test_private_addresses_are_rejected (services.tests_features.JobImportTests.test_private_addresses_are_rejected) ... ok
test_custom_query_leads_the_plan (services.tests_job_search.JobSearchApiTests.test_custom_query_leads_the_plan) ... ok
(services.tests_job_search.JobSearchApiTests.test_featured_is_public_tech_only_and_cached) ... ok
test_keeps_only_recent_searches (services.tests_job_search.JobSearchApiTests.test_keeps_only_recent_searches) ... ok
(services.tests_job_search.JobSearchApiTests.test_other_users_cannot_read_or_use_cv) ... ok
(services.tests_job_search.JobSearchApiTests.test_search_ranks_relevant_offers_and_persists) ... ok
test_overlap_score (services.tests_scoring.ScoringTests.test_overlap_score) ... ok
test_synonyms (services.tests_scoring.ScoringTests.test_synonyms) ... ok
test_cache_avoids_refetching_feeds (services.tests_job_search.CollectTests.test_cache_avoids_refetching_feeds) ... ok
(services.tests_job_search.CollectTests.test_dedupes_and_reports_failing_source) ... Job source arbeitnow failed:
(services.tests_job_search.CollectTests.test_disabled_keyed_sources_are_skipped) ... ok
test_location_filter (services.tests_job_search.ScoringTests.test_location_filter) ... ok
(services.tests_job_search.ScoringTests.test_relevant_offer_outscores_unrelated_one) ... ok
test_skills_require_word_boundaries (services.tests_job_search.ScoringTests.test_skills_require_word_boundaries) ... ok
(services.tests_job_search.ScoringTests.test_title_similarity_handles_aliases_and_compounds) ... ok
Ran 52 tests in 12.856s
OK
```

```
PS C:\resumematch-ai> coverage report --sort=cover

Name                                                Stmts   Miss  Cover
-----------------------------------------------------------------------
cvs\views.py                                           31     17    45%
services\pdf_service.py                                38     18    53%
cvs\serializers.py                                     17      7    59%
accounts\serializers.py                                35     13    63%
analyses\views.py                                     268     95    65%
services\job_import_service.py                        147     37    75%
adminpanel\admin.py                                     9      2    78%
services\job_search_service.py                        234     47    80%
accounts\views.py                                      31      6    81%
analyses\services.py                                  181     32    82%
services\ai_service.py                                428     65    85%
adminpanel\views.py                                   118     12    90%
accounts\models.py                                     11      1    91%
cvs\models.py                                          13      1    92%
analyses\models.py                                     97      6    94%
services\scoring.py                                    57      3    95%
services\ats_service.py                               108      5    95%
adminpanel\models.py                                   22      1    95%
adminpanel\management\commands\seed_demo.py            77      1    99%
accounts\__init__.py                                    0      0   100%
accounts\admin.py                                       8      0   100%
accounts\apps.py                                        4      0   100%
accounts\urls.py                                        3      0   100%
adminpanel\__init__.py                                  0      0   100%
adminpanel\apps.py                                      5      0   100%
adminpanel\demo_data.py                                50      0   100%
adminpanel\management\__init__.py                       0      0   100%
adminpanel\management\commands\__init__.py              0      0   100%
adminpanel\serializers.py                              38      0   100%
adminpanel\urls.py                                      3      0   100%
analyses\__init__.py                                    0      0   100%
analyses\admin.py                                      25      0   100%
analyses\apps.py                                        4      0   100%
analyses\serializers.py                                91      0   100%
analyses\urls.py                                        3      0   100%
cvs\__init__.py                                         0      0   100%
cvs\admin.py                                            6      0   100%
cvs\apps.py                                             4      0   100%
cvs\urls.py                                             3      0   100%
services\__init__.py                                    0      0   100%
-----------------------------------------------------------------------
TOTAL                                                2169    369    83%
```



ResumeMatch AI backend coverage: 83%

Files | Functions | Classes

filter... hide covered

coverage.py v7.16.2, created at 2026-10-11 14:24 +0100

File ▲	statements	missing	excluded	coverage
accounts__init__.py	0	0	0	100%
accounts\admin.py	8	0	0	100%
accounts\apps.py	4	0	0	100%
accounts\models.py	11	1	0	91%
accounts\serializers.py	35	13	0	63%
accounts\urls.py	3	0	0	100%
accounts\views.py	31	6	0	81%
adminpanel__init__.py	0	0	0	100%
adminpanel\admin.py	9	2	0	78%
adminpanel\apps.py	5	0	0	100%
adminpanel\demo_data.py	50	0	0	100%
adminpanel\management__init__.py	0	0	0	100%
adminpanel\management\commands__init__.py	0	0	0	100%
adminpanel\management\commands\seed_demo.py	77	1	0	99%
adminpanel\models.py	22	1	0	95%
adminpanel\serializers.py	38	0	0	100%
adminpanel\urls.py	3	0	0	100%
adminpanel\views.py	118	12	0	90%
analyses__init__.py	0	0	0	100%
analyses\admin.py	25	0	0	100%
analyses\apps.py	4	0	0	100%
analyses\models.py	97	6	0	94%
analyses\serializers.py	91	0	0	100%
analyses\services.py	181	32	0	82%

---

7. Tool 2 — Vitest (frontend unit tests)

Purpose: fast unit tests of the React application's pure logic (helpers, translations), with V8 coverage.

Command

```
 cd frontend
npm run test:coverage
```

Results

Metric	Value
Test files / tests	2 / 19
Passed / failed	**19 / 0**
Duration	0.65 s
Coverage of `src/lib` (business logic)	**90 %** statements · 84.6 % branches · 90 % functions
Coverage of whole `src` (incl. pages/components)	**4.4 %** statements

Tested: score buckets, job-offer text builder, Kanban grouping by status, SVG score-chart geometry and trend, job ranking, URL validation, translation lookup/fallback/interpolation, and **parity of keys and placeholders between French and English**.

Finding: React pages, components and contexts have **no unit tests** (0 %). They are exercised only by the Playwright end-to-end suite (Tool 3).

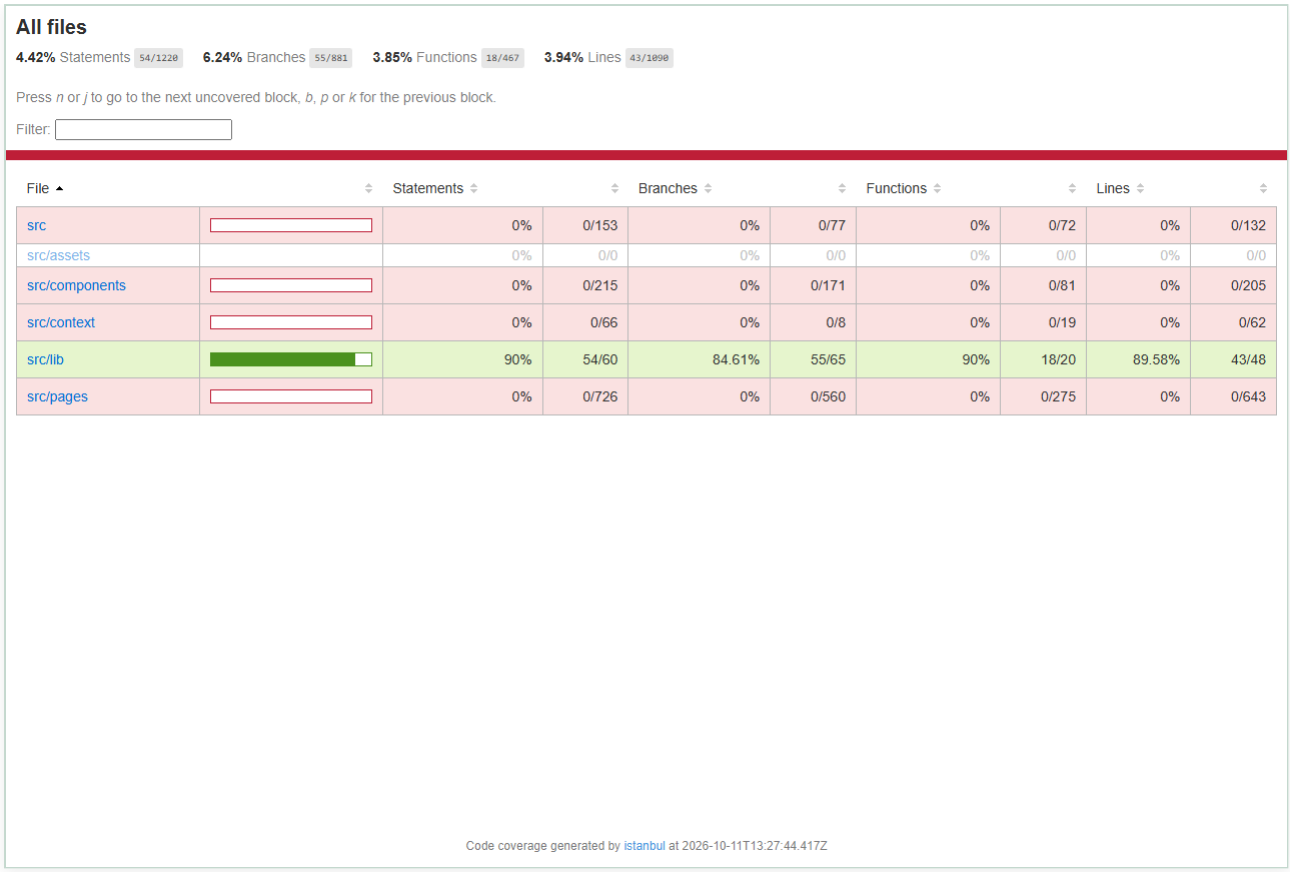
```
PS C:\resumematch-ai> npx vitest run --coverage

 Ôé╝ src/lib/helpers.test.js > scoreTone > buckets scores at 70 and 45 1ms
 Ôé╝ src/lib/helpers.test.js > jobOfferText > prefixes the description with the fields the analyzer parses 0ms
 Ôé╝ src/lib/helpers.test.js > jobOfferText > skips empty fields 0ms
 Ôé╝ src/lib/helpers.test.js > groupByStatus > puts every item in its column and unknown statuses in to_apply 0ms
 Ôé╝ src/lib/helpers.test.js > score chart > maps 100 to the top and 0 to the bottom of the drawing area 0ms
 Ôé╝ src/lib/helpers.test.js > score chart > clamps out-of-range scores and centers a single point 0ms
 Ôé╝ src/lib/helpers.test.js > score chart > builds an SVG path 0ms
 Ôé╝ src/lib/helpers.test.js > score chart > computes the trend between first and last score 0ms
 Ôé╝ src/lib/helpers.test.js > rankJobs > ranks finished jobs by score, unfinished ones last 0ms
 Ôé╝ src/lib/helpers.test.js > misc > sums learning weeks 0ms
 Ôé╝ src/lib/helpers.test.js > misc > only accepts http(s) URLs 0ms
 Ôé╝ src/lib/i18n.test.js > translate > returns the string for the requested language 2ms
 Ôé╝ src/lib/i18n.test.js > translate > interpolates variables and leaves unknown placeholders intact 0ms
 Ôé╝ src/lib/i18n.test.js > translate > falls back to French, then to the key itself 0ms
 Ôé╝ src/lib/i18n.test.js > detectLanguage > prefers a valid stored value, then the browser language, then French 0ms
 Ôé╝ src/lib/i18n.test.js > translations > has the same keys in every language 1ms
 Ôé╝ src/lib/i18n.test.js > translations > keeps the same placeholders across languages 4ms

 Test Files  2 passed (2)
      Tests  19 passed (19)
   Start at  14:27:43
   Duration  649ms (transform 73ms, setup 0ms, import 108ms, tests 15ms, environment 0ms)

 % Coverage report from v8
-------------------|---------|----------|---------|---------|-------------------
File               | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------------|---------|----------|---------|---------|-------------------
All files          |    4.42 |     6.24 |    3.85 |    3.94 |
 src               |       0 |        0 |       0 |       0 |
  App.jsx          |       0 |      100 |       0 |       0 | 13-81
  api.js           |       0 |        0 |       0 |       0 | 1-196
  main.jsx         |       0 |      100 |     100 |       0 | 6
 src/components    |       0 |        0 |       0 |       0 |
  AdminRoute.jsx   |       0 |        0 |       0 |       0 | 5-9
  Navbar.jsx       |       0 |        0 |       0 |       0 | 10-181
  ...UrlImport.jsx |       0 |        0 |       0 |       0 | 7-39
  ...ctedRoute.jsx |       0 |        0 |       0 |       0 | 6-16
  ResultExtras.jsx |       0 |        0 |       0 |       0 | 10-306
  ...eTimeline.jsx |       0 |        0 |       0 |       0 | 8-82
  motion.jsx       |       0 |        0 |       0 |       0 | 12-254
  ui.jsx           |       0 |        0 |       0 |       0 | 7-199
 src/context       |       0 |        0 |       0 |       0 |
  AuthContext.jsx  |       0 |        0 |       0 |       0 | 4-69
  I18nContext.jsx  |       0 |        0 |       0 |       0 | 4-26
  ...usContext.jsx |       0 |      100 |       0 |       0 | 5-29
 src/lib           |      90 |    84.61 |      90 |   89.58 |
  helpers.js       |   97.67 |    93.61 |     100 |     100 | 13,45,58
  i18n.js          |     100 |    84.61 |     100 |     100 | 7,14
  sources.js       |       0 |        0 |       0 |       0 | 1-26
 src/pages         |       0 |        0 |       0 |       0 |
  ...boardPage.jsx |       0 |        0 |       0 |       0 | 12-614
  ...izardPage.jsx |       0 |        0 |       0 |       0 | 12-399
  ...BoardPage.jsx |       0 |        0 |       0 |       0 | 10-120
  ComparePage.jsx  |       0 |        0 |       0 |       0 | 13-190
  ...boardPage.jsx |       0 |        0 |       0 |       0 | 12-129
  HistoryPage.jsx  |       0 |        0 |       0 |       0 | 9-206
  ...earchPage.jsx |       0 |        0 |       0 |       0 | 14-380
  LandingPage.jsx  |       0 |        0 |       0 |       0 | 18-397
  LoginPage.jsx    |       0 |        0 |       0 |       0 | 9-57
  ProfilePage.jsx  |       0 |        0 |       0 |       0 | 8-119
  RegisterPage.jsx |       0 |        0 |       0 |       0 | 9-89
  ResultsPage.jsx  |       0 |        0 |       0 |       0 | 12-312
-------------------|---------|----------|---------|---------|-------------------

============================== Coverage summary ==============================
Statements   : 4.42% ( 54/1220 )
Branches     : 6.24% ( 55/881 )
Functions    : 3.85% ( 18/467 )
Lines        : 3.94% ( 43/1090 )
==============================================================================
```



---

8. Tool 3 — Playwright (end-to-end browser tests)

Purpose: drive a real Chromium browser through complete user journeys against the running frontend + backend, and capture screenshots.

Files: `frontend/playwright.config.js` , `frontend/e2e/app.spec.js`

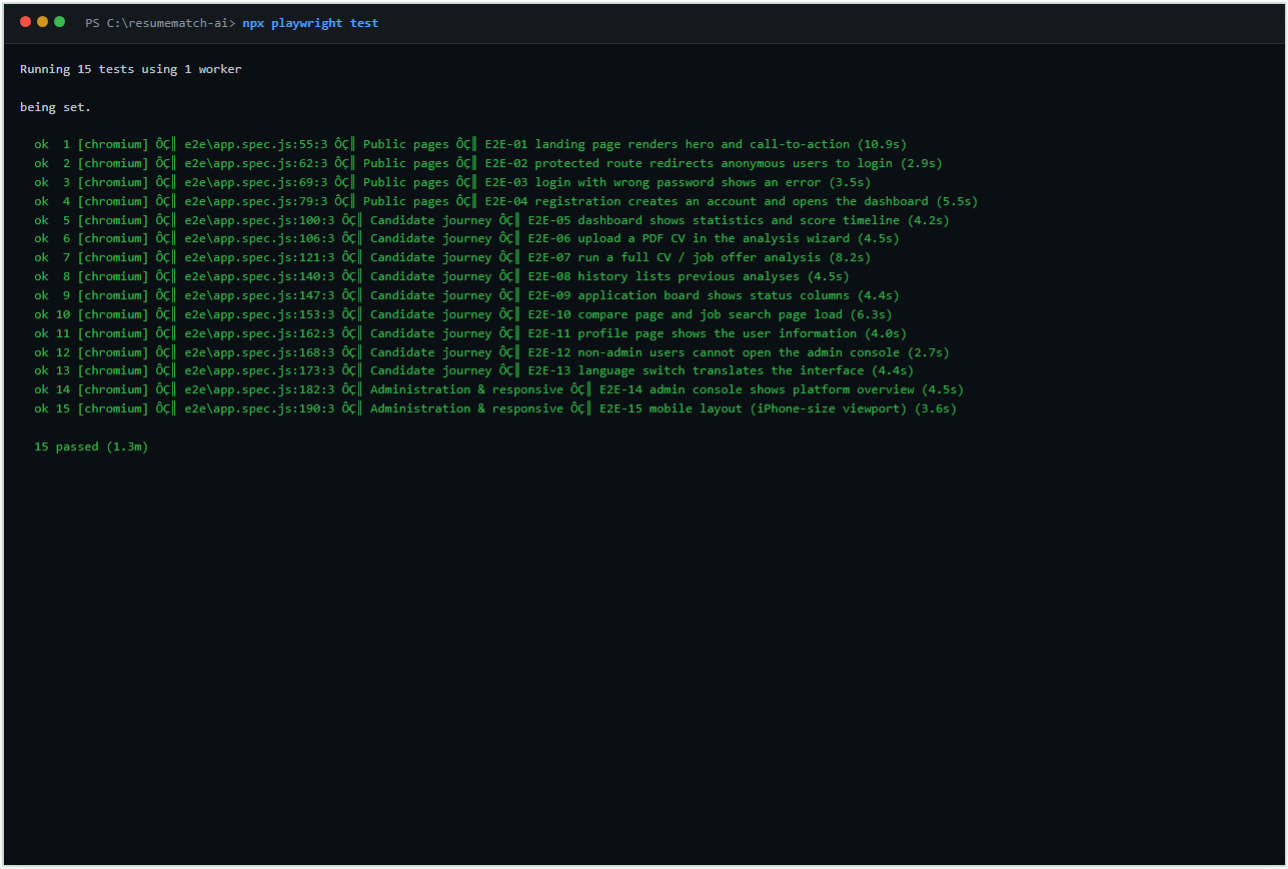
Command

```
 cd frontend
npm run test:e2e
```

Results — 15 / 15 passed in 75 s (Chromium 1280×800, locale fr-FR)

ID	Scenario	Result
E2E-01	Landing page renders hero and call-to-action	✅ Pass
E2E-02	Protected route redirects anonymous users to login	✅ Pass
E2E-03	Login with a wrong password shows an error	✅ Pass
E2E-04	Registration creates an account and opens the dashboard	✅ Pass
E2E-05	Dashboard shows statistics and score timeline	✅ Pass
E2E-06	Upload a PDF CV in the analysis wizard	✅ Pass

ID	Scenario	Result
E2E-07	Full CV / job-offer analysis (4-step wizard → results)	✅ Pass — analysis completed in **148 ms**
E2E-08	History lists previous analyses	✅ Pass
E2E-09	Application board shows status columns	✅ Pass
E2E-10	Compare page and job-search page load	✅ Pass
E2E-11	Profile page shows the user's information	✅ Pass
E2E-12	Non-admin users cannot open the admin console	✅ Pass
E2E-13	Language switch translates the interface (FR → EN)	✅ Pass
E2E-14	Admin console shows the platform overview	✅ Pass
E2E-15	Mobile layout (390×844 viewport)	✅ Pass



Search tests

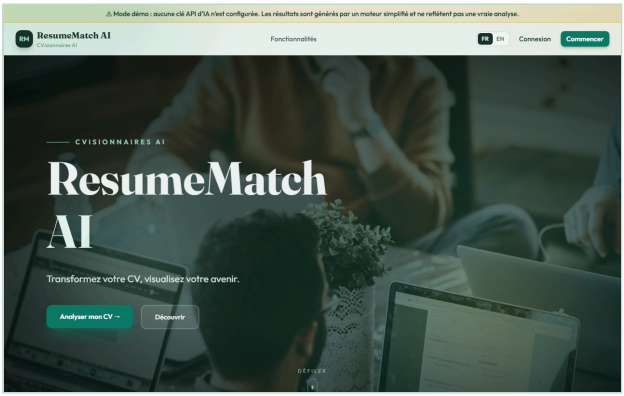
All 15 | Passed 15 | Failed 0 | Flaky 0 | Skipped 0

Project: chromium — 11/10/2026 14:28:59 — Total time: 1.3m

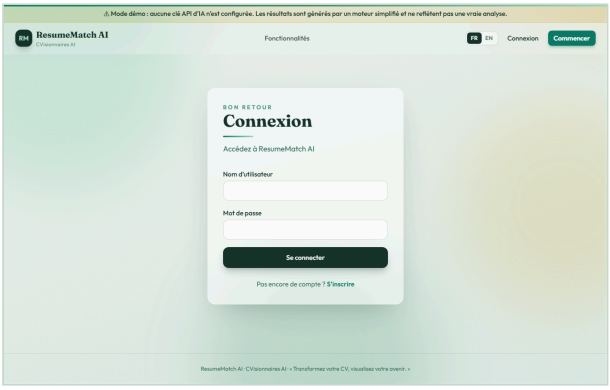
app.spec.js

✓ Public pages › E2E-01 landing page renders hero and call-to-action chromium app.spec.js:55		10.9s
✓ Public pages › E2E-02 protected route redirects anonymous users to login chromium app.spec.js:62		2.9s
✓ Public pages › E2E-03 login with wrong password shows an error chromium app.spec.js:69		3.5s
✓ Public pages › E2E-04 registration creates an account and opens the dashboard chromium app.spec.js:79		5.5s
✓ Candidate journey › E2E-05 dashboard shows statistics and score timeline chromium app.spec.js:100		4.2s
✓ Candidate journey › E2E-06 upload a PDF CV in the analysis wizard chromium app.spec.js:106		4.5s
✓ Candidate journey › E2E-07 run a full CV / job offer analysis chromium app.spec.js:121		8.2s
✓ Candidate journey › E2E-08 history lists previous analyses chromium app.spec.js:140		4.5s
✓ Candidate journey › E2E-09 application board shows status columns chromium app.spec.js:147		4.4s
✓ Candidate journey › E2E-10 compare page and job search page load chromium app.spec.js:153		6.3s
✓ Candidate journey › E2E-11 profile page shows the user information chromium app.spec.js:162		4.0s
✓ Candidate journey › E2E-12 non-admin users cannot open the admin console chromium app.spec.js:168		2.7s

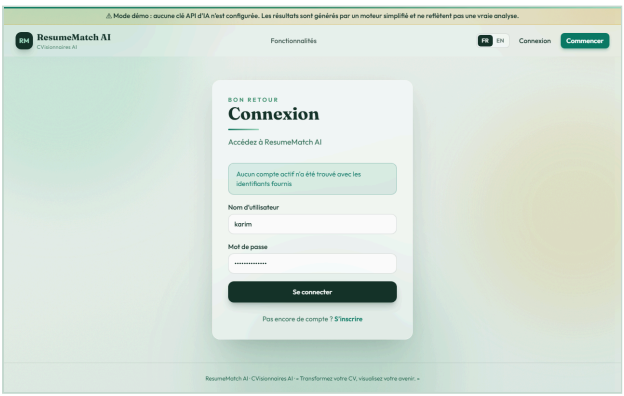
8.1 Screenshots captured during the end-to-end run



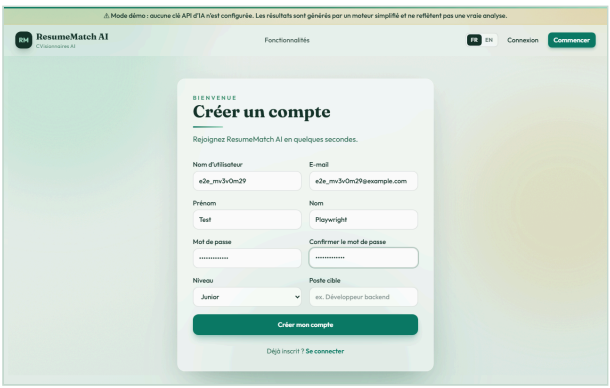
E2E-01 Landing page



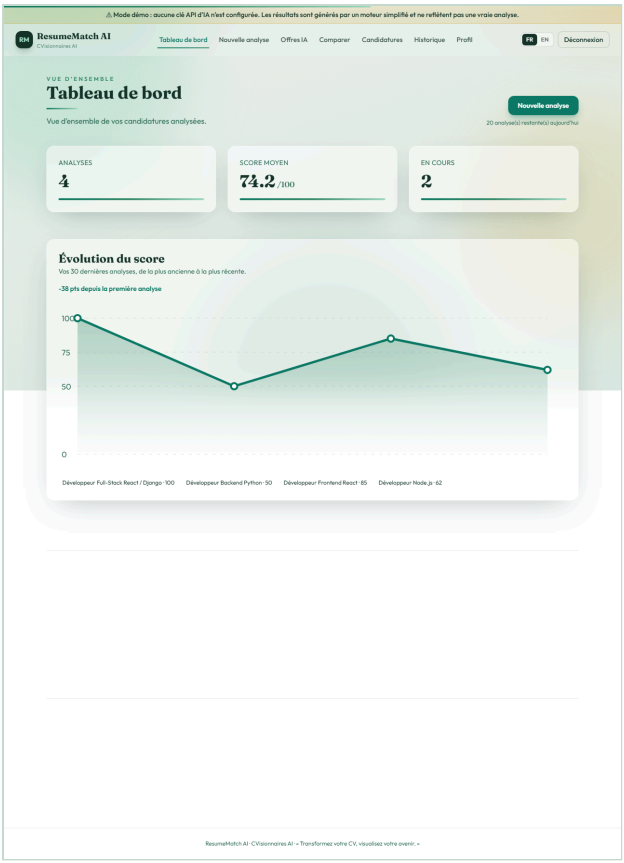
E2E-02 Redirect to login



E2E-03 Wrong password



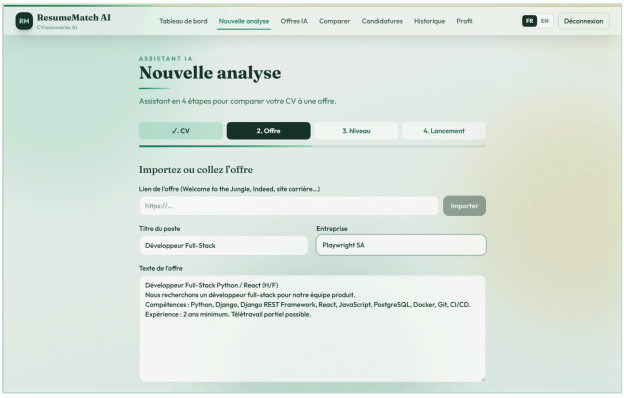
E2E-04 Registration form



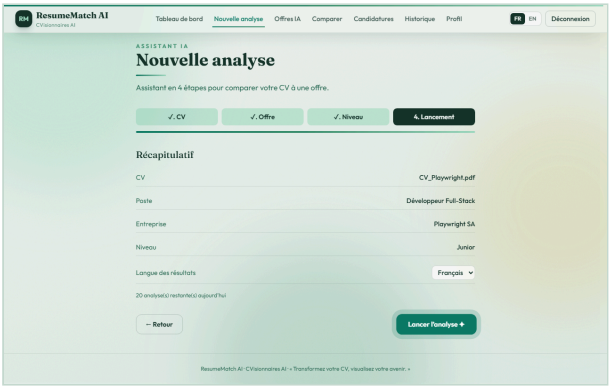
E2E-05 Dashboard



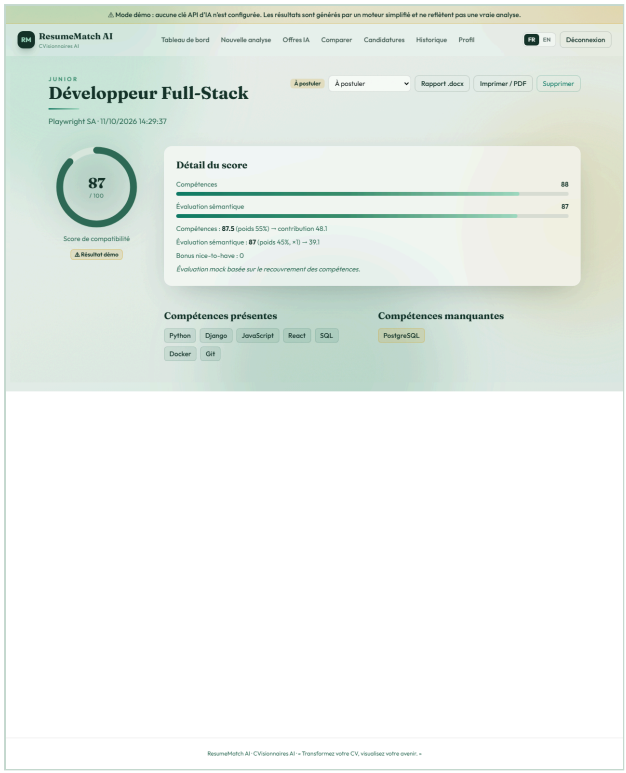
E2E-06 CV upload



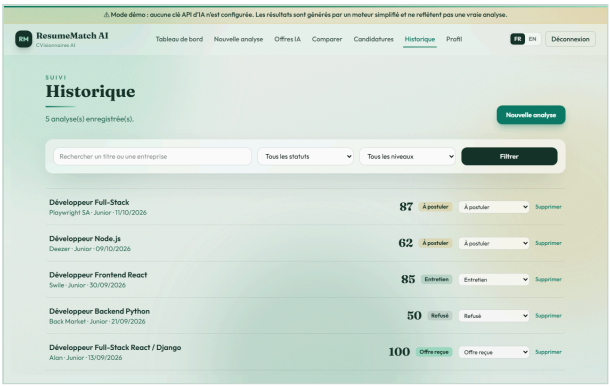
E2E-07 Wizard — job offer



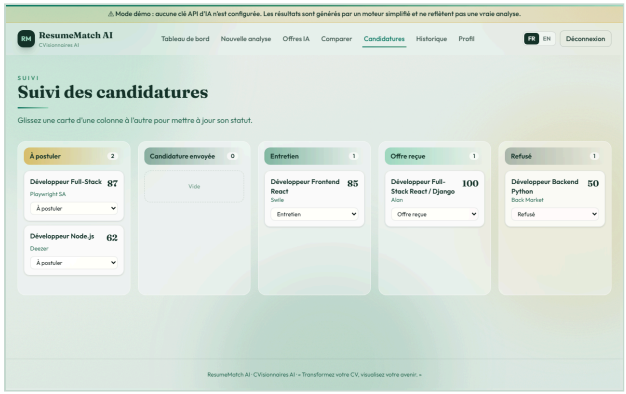
E2E-07 Wizard — summary



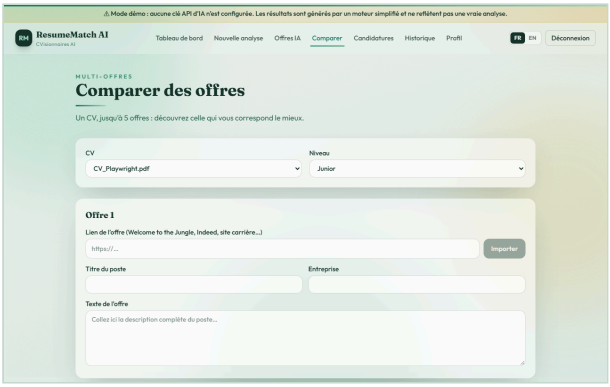
E2E-07 Analysis result (score 87/100)



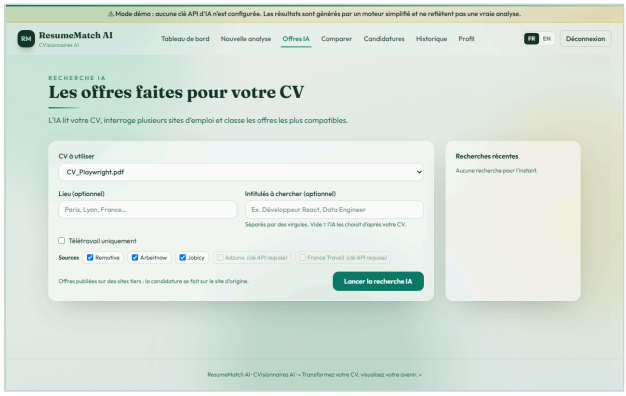
E2E-08 History



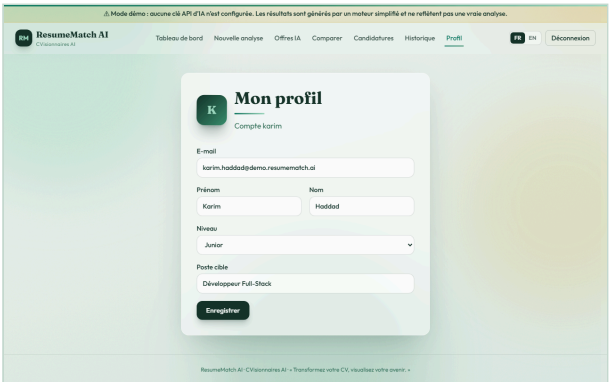
E2E-09 Application board



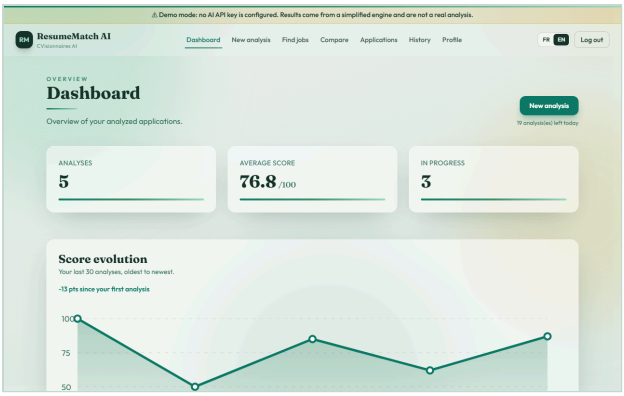
E2E-10 Compare offers



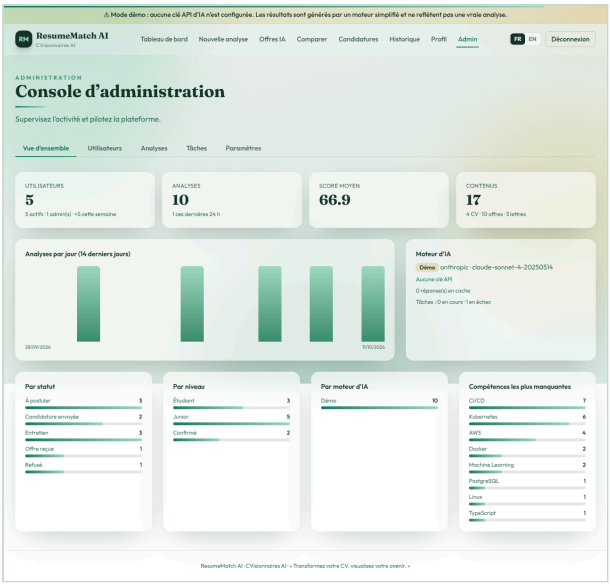
E2E-10 AI job search



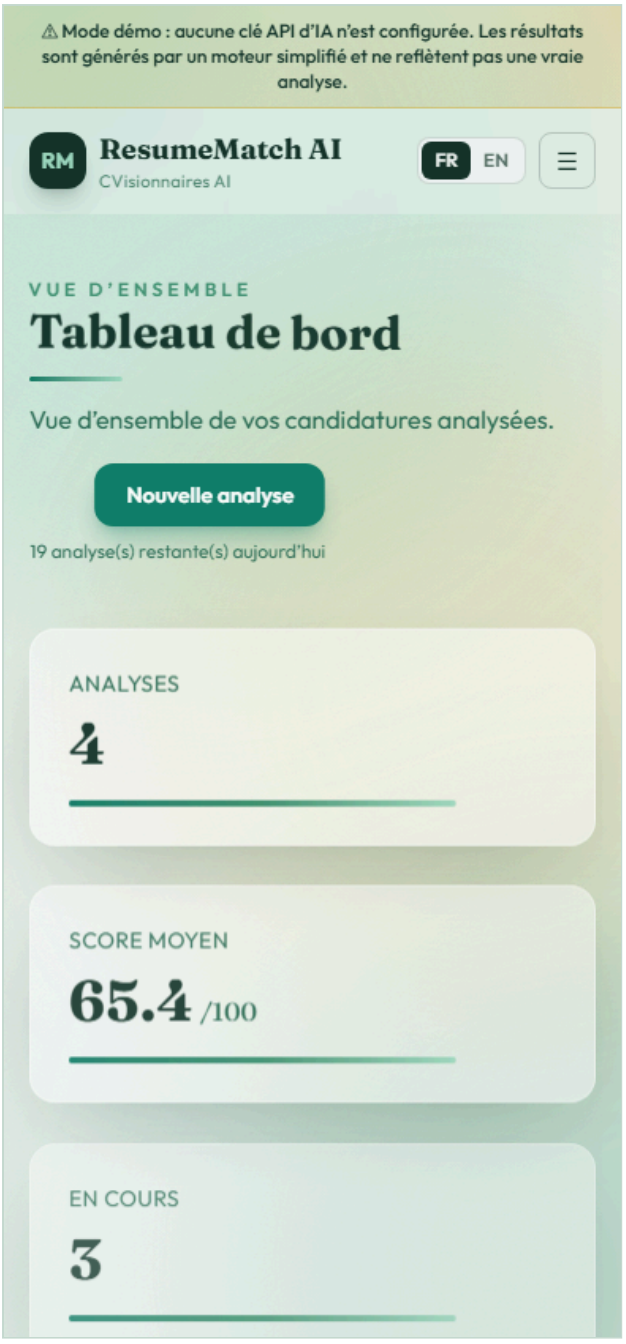
E2E-11 Profile



E2E-13 English interface



E2E-14 Admin console



E2E-15 Mobile dashboard

Observation (UX): on E2E-03 the error message is displayed in the brand teal colour. Since the palette change, the `coral` colour token used by `Alert type="error"` is teal, so errors are not visually distinguishable from informational messages.

---

9. Tool 4 — Locust (load and stress testing)

Purpose: simulate many concurrent candidates using the REST API and measure throughput, latency percentiles and error rate.

File: `backend/loadtests/locustfile.py` — each virtual user logs in (JWT) and then randomly, with weights, opens the dashboard (5), lists analyses (4), opens an analysis (3), reads the profile (3), checks

system status (2), lists job sources (2), **runs a new analysis** (1) and calls the health check (1), waiting 0.5–2 s between actions.

Commands

```
 cd backend
locust -f loadtests/locustfile.py --host http://127.0.0.1:8765 --headless -u 20  -r 5  -t 60s -
locust -f loadtests/locustfile.py --host http://127.0.0.1:8765 --headless -u 100 -r 10 -t 60s -
```

9.1 Normal load — 20 concurrent users, 60 s

Endpoint	Requests	Failures	Median (ms)	Avg (ms)	p95 (ms)	Max (ms)
GET /api/dashboard/stats/	238	0	10	11.7	20	68
GET /api/analyses/	199	0	7	8.2	13	23
GET /api/analyses/:id/	129	0	5	6.1	10	32
GET /api/auth/me	130	0	4	4.9	9	19
GET /api/job-search/sources	97	0	4	4.1	7	9
GET /api/system/status/	71	0	6	6.9	12	13
GET /api/health/	40	0	2	2.5	5	17
GET /api/cvs/	20	0	7	7.1	14	14
POST /api/analyses/	31	0	23	23.4	30	31
POST /api/auth/login	20	0	770	788	950	953
Total	**975**	**0 (0 %)**	**7**	**24**	**23**	**953**

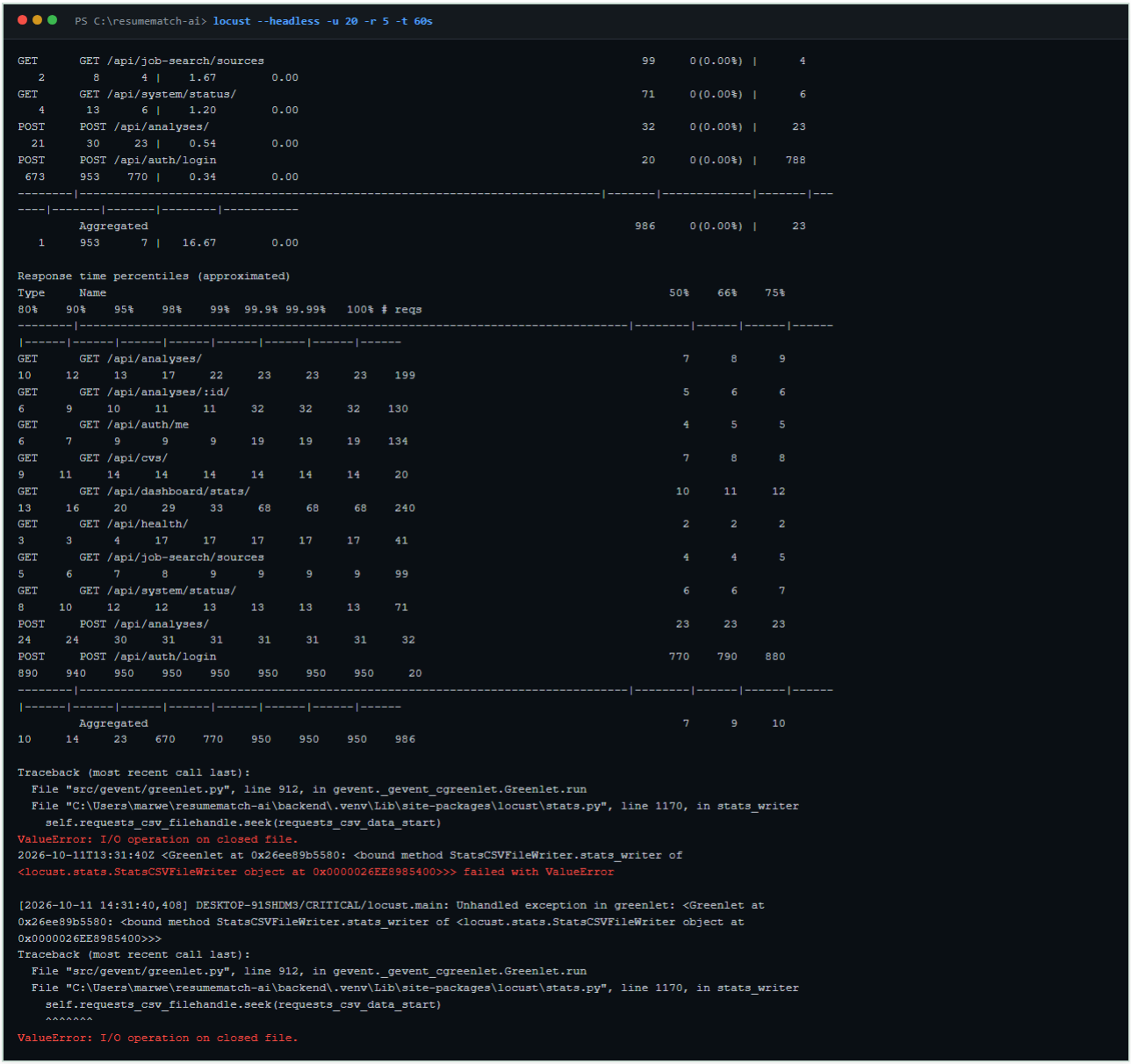
Throughput: **16.7 requests/s** (limited by the simulated think time, not by the server).

9.2 Stress — 100 concurrent users, 60 s

Metric	Value
Requests	4 517
Throughput	**76.2 requests/s**
Median / p95 / p99	10 ms / 97 ms / 1 800 ms
Login median under stress	1 800 ms (max 2.6 s)
Failures	153 (3.4 %) — **all `429 Too Many Requests` on `POST /api/analyses/`**
Server errors (5xx) / timeouts	**0**

Interpretation of the 429 responses: the 100 virtual users share the 3 demo accounts, so each account quickly exceeded the business rule *"20 analyses per user per 24 h"* (`ANALYSIS_DAILY_QUOTA`) and the AI rate limit (30/hour). The API correctly refused the extra analyses with HTTP 429 and a clear message, which is the expected behaviour. Read-only endpoints served every request.

Finding: login is the slowest operation (≈0.8 s alone, 1.8 s under stress). This is caused by the deliberately expensive password hashing (PBKDF2) running on a single development server process; it is a security feature, but it limits how many logins per second one process can handle.



```
PS C:\resumematch-ai> locust --headless -u 20 -r 5 -t 60s

GET      GET /api/job-search/sources                                                    99     0(0.00%) |      4
   2       8      4 |    1.67        0.00
GET      GET /api/system/status/                                                        71     0(0.00%) |      6
   4      13      6 |    1.20        0.00
POST     POST /api/analyses/                                                            32     0(0.00%) |     23
  21      30     23 |    0.54        0.00
POST     POST /api/auth/login                                                           20     0(0.00%) |    788
 673     953    770 |    0.34        0.00
--------|------------------------------------------------------------------------------|-------|-------------|-------|---
----|-------|-------|--------|-----------
         Aggregated                                                                   986     0(0.00%) |     23
   1     953      7 |   16.67        0.00

Response time percentiles (approximated)
Type     Name                                                                                50%    66%    75%
80%    90%    95%    98%    99%  99.9% 99.99%   100% # reqs
--------|------------------------------------------------------------------------------|--------|------|------|------
|------|------|------|------|------|------|------|------
GET      GET /api/analyses/                                                                   7      8      9
10     12     13     17     22     23     23     23    199
GET      GET /api/analyses/:id/                                                               5      6      6
6      9     10     11     11     32     32     32    130
GET      GET /api/auth/me                                                                     4      5      5
6      7      9      9      9     19     19     19    134
GET      GET /api/cvs/                                                                        7      8      8
9     11     14     14     14     14     14     14     20
GET      GET /api/dashboard/stats/                                                           10     11     12
13     16     20     29     33     68     68     68    240
GET      GET /api/health/                                                                     2      2      2
3      3      4     17     17     17     17     17     41
GET      GET /api/job-search/sources                                                          4      4      5
5      6      7      8      9      9      9      9     99
GET      GET /api/system/status/                                                              6      6      7
8     10     12     12     13     13     13     13     71
POST     POST /api/analyses/                                                                 23     23     23
24     24     30     31     31     31     31     31     32
POST     POST /api/auth/login                                                               770    790    880
890    940    950    950    950    950    950    950     20
--------|------------------------------------------------------------------------------|--------|------|------|------
|------|------|------|------|------|------|------|------
         Aggregated                                                                          7      9     10
10     14     23    670    770    950    950    950    986

Traceback (most recent call last):
  File "src/gevent/greenlet.py", line 912, in gevent._gevent_cgreenlet.Greenlet.run
  File "C:\Users\marwe\resumematch-ai\backend\.venv\Lib\site-packages\locust\stats.py", line 1170, in stats_writer
    self.requests_csv_filehandle.seek(requests_csv_data_start)
ValueError: I/O operation on closed file.
2026-10-11T13:31:40Z <Greenlet at 0x26ee89b5580: <bound method StatsCSVFileWriter.stats_writer of
<locust.stats.StatsCSVFileWriter object at 0x0000026EE8985400>>> failed with ValueError

[2026-10-11 14:31:40,408] DESKTOP-91SHDM3/CRITICAL/locust.main: Unhandled exception in greenlet: <Greenlet at
0x26ee89b5580: <bound method StatsCSVFileWriter.stats_writer of <locust.stats.StatsCSVFileWriter object at
0x0000026EE8985400>>>
Traceback (most recent call last):
  File "src/gevent/greenlet.py", line 912, in gevent._gevent_cgreenlet.Greenlet.run
  File "C:\Users\marwe\resumematch-ai\backend\.venv\Lib\site-packages\locust\stats.py", line 1170, in stats_writer
    self.requests_csv_filehandle.seek(requests_csv_data_start)
    ^^^^^^^
ValueError: I/O operation on closed file.
```

Locust Test Report

During: 11/10/2026 14:30:40 - 11/10/2026 14:31:39 (59 seconds)
Target Host: http://127.0.0.1:8765
Script: locustfile.py

Request Statistics

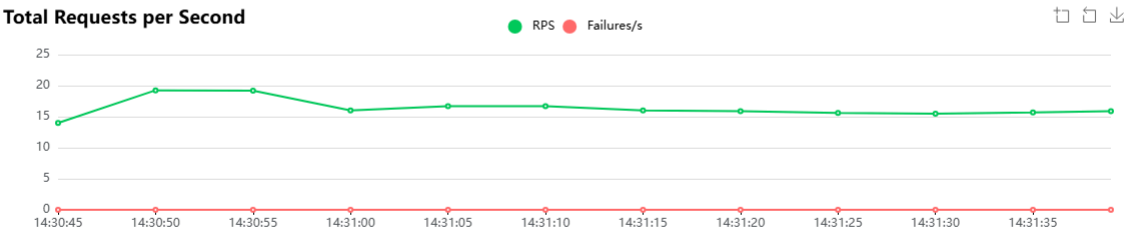
Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /api/analyses/	199	0	8.21	5	23	14419.21	3.37	0
GET	GET /api/analyses/:id/	130	0	6.09	4	32	2125.12	2.2	0
GET	GET /api/auth/me	134	0	4.84	3	19	232.9	2.27	0
GET	GET /api/cvs/	20	0	7.13	4	14	1522.1	0.34	0
GET	GET /api/dashboard/stats/	240	0	11.7	7	68	10408.88	4.06	0
GET	GET /api/health/	41	0	2.52	1	17	45	0.69	0
GET	GET /api/job-search/sources	99	0	4.15	3	9	519	1.67	0
GET	GET /api/system/status/	71	0	6.87	4	13	153.73	1.2	0
POST	POST /api/analyses/	32	0	23.35	22	31	1848.22	0.54	0

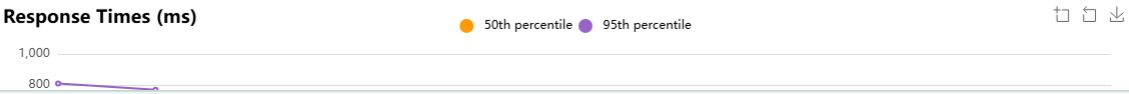
GET	GET /api/system/status/	6	6	7	8	10	12	13	13
POST	POST /api/analyses/	23	23	23	24	24	30	31	31
POST	POST /api/auth/login	770	790	810	890	940	950	950	950
	Aggregated	7	8	9	10	14	23	770	950

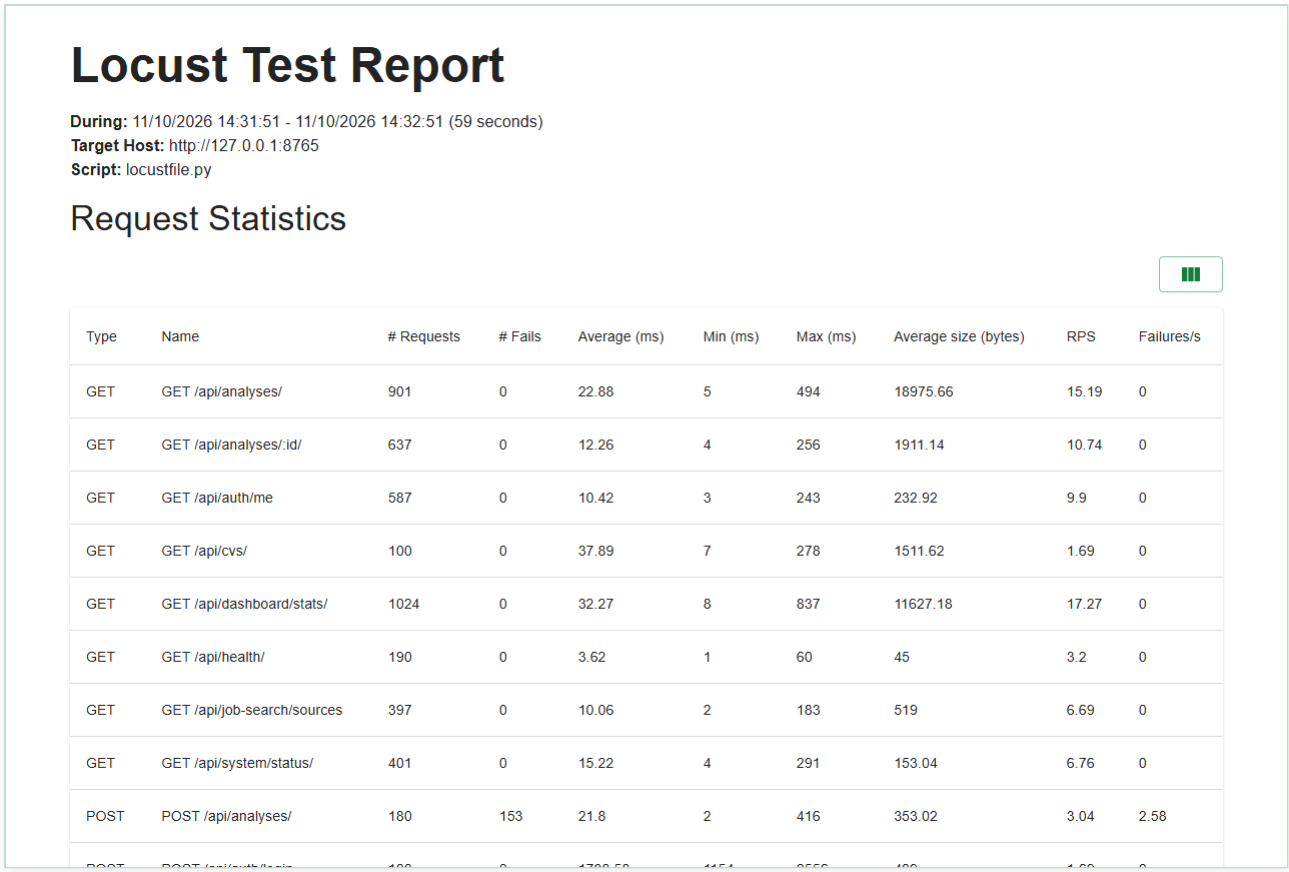
Failures Statistics

# Failures	Method	Name	Message	First Seen	Last Seen

Charts







Locust Test Report

During: 11/10/2026 14:31:51 - 11/10/2026 14:32:51 (59 seconds)
Target Host: http://127.0.0.1:8765
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /api/analyses/	901	0	22.88	5	494	18975.66	15.19	0
GET	GET /api/analyses/:id/	637	0	12.26	4	256	1911.14	10.74	0
GET	GET /api/auth/me	587	0	10.42	3	243	232.92	9.9	0
GET	GET /api/cvs/	100	0	37.89	7	278	1511.62	1.69	0
GET	GET /api/dashboard/stats/	1024	0	32.27	8	837	11627.18	17.27	0
GET	GET /api/health/	190	0	3.62	1	60	45	3.2	0
GET	GET /api/job-search/sources	397	0	10.06	2	183	519	6.69	0
GET	GET /api/system/status/	401	0	15.22	4	291	153.04	6.76	0
POST	POST /api/analyses/	180	153	21.8	2	416	353.02	3.04	2.58

10. Tool 5 — Lighthouse (performance, accessibility, best practices, SEO)

Purpose: audit the production build (`vite build` + `vite preview`) of the public pages with Google Lighthouse 12.8.2 (headless Chromium).

Command (example)

```
npx lighthouse http://127.0.0.1:43131/ --preset=desktop --output=html --output=json `
  --output-path=docs/test-report/lighthouse/landing-desktop `
  --only-categories=performance,accessibility,best-practices,seo
```

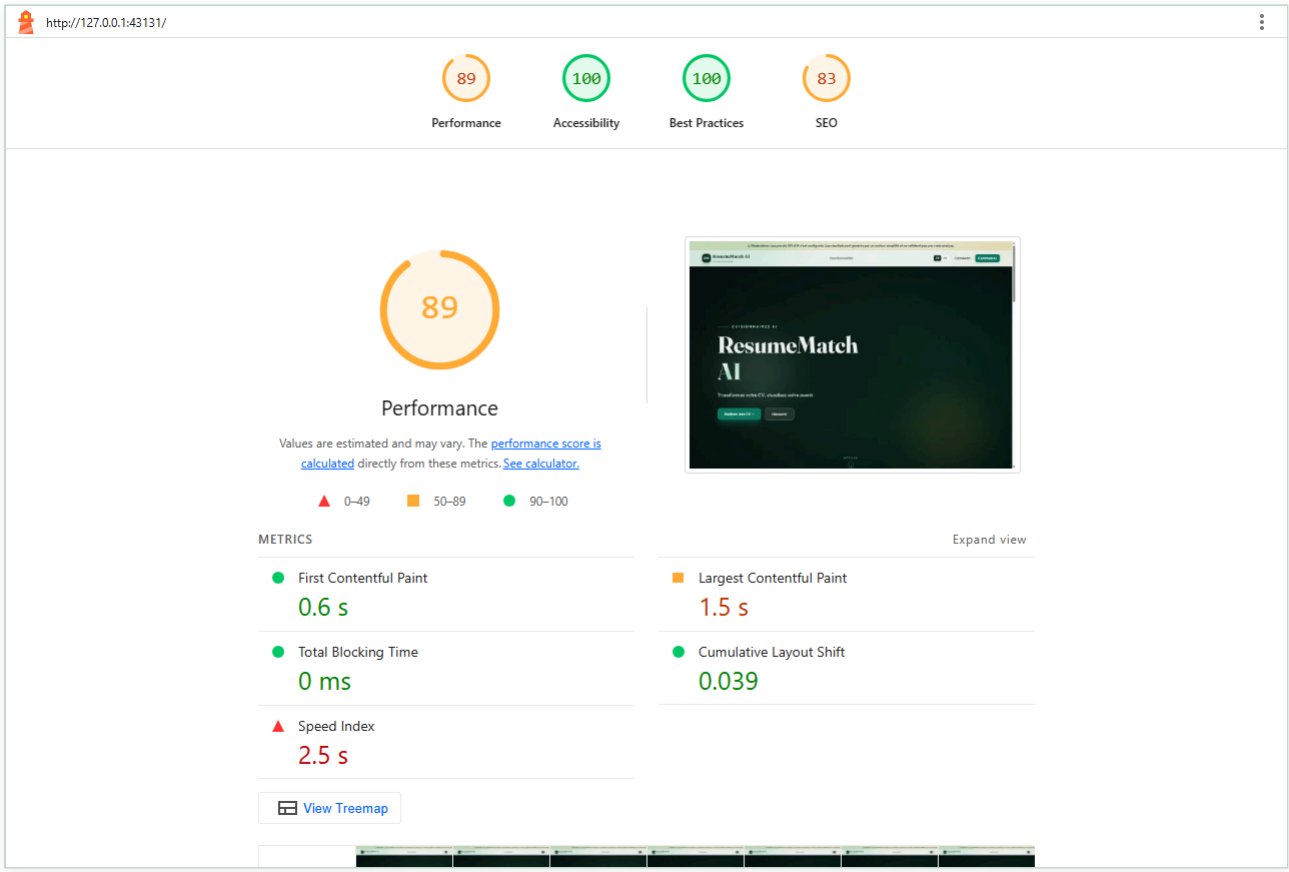
Results

Page	Device	Performance	Accessibility	Best practices	SEO	FCP	LCP	TBT	CLS	Speed Index
Landing `/`	Desktop	**89**	**100**	**100**	83	0.6 s	1.5 s	0 ms	0.039	2.5 s
Landing `/`	Mobile	**65**	**100**	**100**	83	3.3 s	7.1 s	170 ms	0.034	5.0 s
Login `/connexion`	Desktop	**95**	**100**	**100**	82	0.9 s	1.3 s	0 ms	0.023	1.1 s

Page	Device	Performance	Accessibility	Best practices	SEO	FCP	LCP	TBT	CLS	Speed Index
Register `/inscription`	Desktop	**98**	**100**	**100**	82	0.6 s	1.1 s	0 ms	0.023	0.9 s

Findings

- **Mobile LCP 7.1 s:** the largest element is the full-screen hero background, a 1920 px image loaded from `images.unsplash.com` (external host, not optimised for mobile).
- **Main-thread work 2.2 s** and **62 KiB of unused JavaScript** on mobile (animation library + shared bundle).
- **SEO:** the page has no `<meta name="description">`, and `/robots.txt` is missing (the SPA returns `index.html` instead).
- **Accessibility 100 / Best practices 100** on every page: correct contrast, labels, roles, HTTPS-ready headers, no console errors.



http://127.0.0.1:43131/

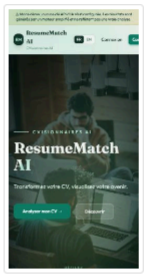
65	100	100	83
Performance	Accessibility	Best Practices	SEO

65

Performance

Values are estimated and may vary. The performance score is calculated directly from these metrics. See calculator.

▲ 0–49 ■ 50–89 ● 90–100



METRICS Expand view

▲ First Contentful Paint 3.3 s	▲ Largest Contentful Paint 7.1 s
● Total Blocking Time 170 ms	● Cumulative Layout Shift 0.034
■ Speed Index 5.0 s	

View Treemap

http://127.0.0.1:43131/connexion

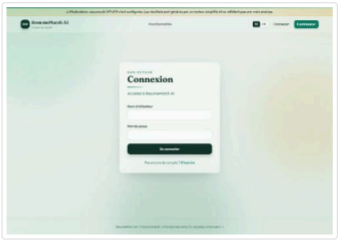
95	100	100	82
Performance	Accessibility	Best Practices	SEO

95

Performance

Values are estimated and may vary. The performance score is calculated directly from these metrics. See calculator.

▲ 0–49 ■ 50–89 ● 90–100



METRICS Expand view

● First Contentful Paint 0.9 s	■ Largest Contentful Paint 1.3 s
● Total Blocking Time 0 ms	● Cumulative Layout Shift 0.023
● Speed Index 1.1 s	

View Treemap

11. Comparison of the five tools

11.1 Tool characteristics

Criterion	Django test + Coverage.py	Vitest	Playwright	Locust	Lighthouse
Test type	Unit + integration	Unit	End-to-end (system)	Load / stress	Quality audit
Layer tested	Backend (API, models, services)	Frontend logic	Whole stack via the browser	Backend API	Frontend (rendered pages)
Language	Python	JavaScript	JavaScript	Python	CLI (no code)
Main metric	Pass/fail + % statements covered	Pass/fail + % coverage	Pass/fail + screenshots	Latency percentiles, RPS, error rate	0–100 scores + Web Vitals
Speed	12.9 s	0.65 s	75 s	60 s per scenario	≈20 s per page
Needs running servers	No (test DB)	No	Yes (frontend + backend)	Yes (backend)	Yes (built frontend)
Detects	Logic bugs, permission and validation errors, regressions	Logic bugs in helpers / i18n	Broken journeys, routing, UI integration, visual issues	Bottlenecks, slow endpoints, errors under load	Slow loading, a11y, SEO, bad practices
Does not detect	UI problems, performance	UI rendering, API issues	Performance limits, untested branches	Functional correctness of UI	Functional bugs
Effort to write tests	Medium	Low	Medium–high	Low–medium	None
Report produced	Console + HTML coverage	Console + HTML coverage	HTML report + JSON + screenshots	HTML report + CSV	HTML + JSON
Best use in CI	Every commit	Every commit	Before merge / release	Before release	Before release

11.2 Results obtained

Tool	Tests / scenarios	Passed	Key result	Rating
Django test + Coverage.py	52 tests	52 (100 %)	83 % backend coverage	✅ Very good
Vitest	19 tests	19 (100 %)	90 % of `src/lib`, 4.4 % of all frontend code	⚠️ Good logic coverage, no component tests
Playwright	15 journeys	15 (100 %)	All user journeys work; analysis in 148 ms	✅ Very good

Tool	Tests / scenarios	Passed	Key result	Rating
Locust	2 scenarios (20 and 100 users)	0 server errors	Median 7–10 ms, 76 RPS at 100 users; login 0.8–1.8 s	✅ Good (login is the bottleneck)
Lighthouse	4 audits	—	A11y 100, BP 100, Desktop perf 89–98, Mobile perf 65, SEO 82–83	⚠️ Good on desktop, mobile to improve

12. Defects and recommendations

#	Severity	Finding	Source tool	Recommendation
1	Medium	Mobile landing page LCP 7.1 s (performance 65)	Lighthouse	Self-host the hero image in WebP/AVIF with responsive `srcset` (≈800 px for mobile) and `fetchpriority="high"`; preload it
2	Medium	React pages/components have 0 % unit coverage	Vitest	Add component tests with `@testing-library/react` (login form, wizard steps, results page)
3	Medium	Login takes 0.8 s alone and 1.8 s under 100 users	Locust	Run Django with several workers in production (gunicorn/uvicorn); keep refresh tokens so users rarely log in again
4	Low	No meta description, missing `robots.txt` (SEO 82–83)	Lighthouse	Add `<meta name="description">` in `index.html` and `public/robots.txt`
5	Low	62 KiB unused JS, 2.2 s main-thread work on mobile	Lighthouse	Keep route-level code splitting; lazy-load the landing-page animations
6	Low	Error alerts use the brand teal colour	Playwright (screenshot E2E-03)	Give `Alert type="error"` a dedicated red/orange token
7	Low	`cvs/views.py` (45 %) and `pdf_service.py` (53 %) weakly covered	Coverage.py	Add tests for CV upload errors (non-PDF, oversize, empty text) and PDF extraction fallbacks
8	Info	Daily quota correctly returned HTTP 429 under stress	Locust	No action — expected behaviour; document it for load tests

13. Conclusion

Across **86 automated functional tests** and **4 517 load-test requests**, ResumeMatch AI showed **no functional failure and no server error**. The backend is well covered (83 %) and very fast (median 7–10 ms per API call), every user journey works in a real browser on desktop and mobile, and the UI scores 100 in accessibility and best practices.

The five tools turned out to be complementary: **Coverage.py and Vitest** guarantee correctness of the code, **Playwright** proves that the integrated product works for a user, **Locust** shows how it behaves at

scale, and **Lighthouse** evaluates the experience perceived in the browser. No single tool would have found all eight findings in section 12.

Priority next steps: optimise the mobile hero image, add React component tests, and deploy the API with multiple worker processes.

Appendix A — How to reproduce

```
 # Backend tools
cd backend
pip install -r requirements-dev.txt
python -m coverage run --source=accounts,cvs,analyses,adminpanel,services --omit="*/migrations/*
python -m coverage report

# Frontend unit tests
cd ../frontend
npm install
npm run test:coverage

# Start the application with demo data (two terminals)
cd ../backend; python manage.py seed_demo --reset; python manage.py runserver 127.0.0.1:8765
cd ../frontend; npx vite --port 43130 --strictPort

# End-to-end
npx playwright install chromium
npm run test:e2e

# Load test
cd ../backend
locust -f loadtests/locustfile.py --host http://127.0.0.1:8765 --headless -u 20 -r 5 -t 60s

# Lighthouse (on the production build)
cd ../frontend; npm run build; npx vite preview --port 43131
npx lighthouse http://127.0.0.1:43131/ --preset=desktop --view
```

Appendix B — Deliverables in `docs/test-report/`

Path	Content
`TECHNICAL_REPORT.md` / `TECHNICAL_REPORT.docx`	This report
`screenshots/`	32 screenshots (18 from the end-to-end run, 14 of tool reports)
`coverage-backend-html/`	Coverage.py HTML report

Path	Content
`coverage-frontend-html/`	Vitest V8 coverage HTML report
`playwright-html/`	Playwright HTML report
`locust-report.html`, `locust-stress-report.html`	Locust reports
`lighthouse/`	Lighthouse HTML + JSON reports (4 audits)
`raw/`	Raw console logs, Locust CSV, Playwright JSON
`console/`, `tools/`	Console captures and the scripts that render them and build the .docx / .html / .pdf reports
`../conception/diagrammes/`, `../conception/images/`	PlantUML sources and PNG / SVG renderings of the 6 UML diagrams (section 3)
`../conception/PROMPTS.md`, `../conception/CONCEPTION.md`	Full French versions of the prompts and design documents (also as .docx / .pdf)